# Zemlyanika – Module-LWE based KEM with the power-of-two modulus, explicit rejection and revisited decapsulation failures

A.S. Zelenetsky[1, 2, *] and P.G. Klyucharev[2]

[1]QApp, Bolshoy blvd 30b1, Moscow, 121205, Russia
[2]Bauman Moscow State Technical University, 2-nd Baumanskaya, 5, Moscow, 105005, Russia
[*]Author's Email: azelenetskiy@qapp.tech

July 29, 2025

**Abstract**

This work introduces Zemlyanika, a post-quantum, IND-CCA secure key encapsulation mechanism based on the Module-LWE problem. Zemlyanika adopts a standard high-level design approach in which a passively secure public-key encryption scheme is upgraded to an actively secure key encapsulation mechanism via the Fujisaki–Okamoto transform. The scheme is characterized by three key design choices: a power-of-two modulus, explicit rejection, and revised bounds on the decapsulation failure probability.

Employing a power-of-two modulus is uncommon in Module-LWE-based schemes, primarily due to the inapplicability of the Number Theoretic Transform (NTT). However, we argue that this choice offers several often underestimated advantages. We also use explicit rejection, which is more efficient than its implicit counterpart. Recent research shows that both approaches offer comparable security guarantees, and thus this choice does not compromise the scheme's overall security. Finally, we provide a rigorous analysis showing that the conventional upper bound on the decapsulation failure probability can be safely relaxed without weakening security. Increasing this bound yields measurable gains in both performance and resistance to Module-LWE–based attacks.

**Keywords:** key encapsulation mechanism, M-LWE, explicit rejection, decapsulation failure.

# 1 Introduction

Today, lattice-based cryptography offers the best trade-off between performance and the sizes of public keys and ciphertexts among all known post-quantum cryptographic approaches. Its security relies on hard geometric problems over lattices, which are believed to remain computationally hard even for quantum computers. Key encapsulation mechanisms (KEMs) based on structured variants of the Learning With Errors (LWE) [41] and Learning With Rounding (LWR) [7] problems are among the most promising primitives in lattice-based cryptography. For instance, the Kyber KEM [17] and the Dilithium digital signature scheme [18], recently standardized as FIPS 203 [39] and FIPS 204 [38], respectively, are based on the Module-LWE (M-LWE) problem [32]. M-LWE is a structured variant of the classical LWE problem. The additional module structure does not appear to significantly reduce the problem's hardness, while offering improved efficiency. Many modern lattice-based KEMs, including Kyber [17], Saber [21], Smaug-T [19], and Tyber [33], are all based on module versions of the LWE or LWR problems. These KEMs follow a common design paradigm: a passively secure public-key encryption (PKE) scheme is transformed into an actively secure KEM using on of the variant of the Fujisaki–Okamoto (FO) transform [24, 26]. Moreover, the underlying PKE schemes are based on the LPR construction proposed in [34]. As a result, existing M-LWE/M-LWR-based KEMs share the same high-level design framework, differing in parameters such as the modulus, cyclotomic polynomial, noise and secret distributions, and the particular variant of the FO transformation applied.

## 1.1 Our contributions

In this work, we propose an M-LWE-based key encapsulation mechanism (KEM) named **Zemlyanika** (rus. Земляника — wild strawberry). Our construction is characterized by three main features: the use of a power-of-two modulus, explicit rejection, and revised requirements for the decapsulation failure probability.

**Power-of-two modulus.** Zemlyanika is, to the best of our knowledge, the first M-LWE-based KEM that employs a power-of-two modulus. Typically, KEMs based on structured variants of the LWE problem employ a prime modulus, primarily to enable the use of the Number Theoretic Trans-

form (NTT) for fast polynomial multiplication. Since polynomial multiplication significantly affects the overall performance of lattice-based KEMs, prime moduli have traditionally been the preferred choice. However, we argue that the performance benefits of using a power-of-two modulus are often underestimated. In particular, both matrix generation and modular reduction algorithms become substantially more efficient when the modulus is a power-of-two. Moreover, Zemlyanika employs the Toom–Cook algorithm for polynomial multiplication, which, in our implementation, achieves performance comparable to that of NTT-based approaches. It is worth noting that Saber and Smaug-T also employ a power-of-two modulus. Saber is based on M-LWR and therefore employs a power-of-two modulus to simplify the rounding operation. In contrast, Zemlyanika is based on the M-LWE problem, which enables the use of a smaller modulus and, consequently, more efficient Toom–Cook multiplication. Smaug-T adopts a hybrid approach: its public key pseudorandomness relies on the hardness of M-LWE, whereas the ciphertext security is based on M-LWR. Additionally, Smaug-T employs a sparse secret and a custom multiplication algorithm. By contrast, Zemlyanika based only on M-LWE and doesn't employ a sparse secret, which makes it more flexible and provides stronger security assumptions.

**Explicit rejection.** The possibility of decapsulation failure for honestly generated ciphertexts is a notable drawback of all M-LWE/M-LWR-based KEMs. There are two common approaches to handling decapsulation failures: implicit rejection and explicit rejection. All previously proposed M-LWE/M-LWR-based KEMs employ variants of the Fujisaki–Okamoto (FO) transform that use implicit rejection. This choice is primarily motivated by the availability of security proofs for these variants in the Quantum Random Oracle Model (QROM). However, in practice, there is no evidence that explicit rejection is less secure than implicit rejection. Moreover, explicit rejection reduces the number of hashing operations and calls to the random number generator, resulting in improved efficiency. A security proof for the FO transform with explicit rejection in the QROM was established in [28]. Moreover, the recent work [29] demonstrates that these two types of rejection are equivalent in terms of security. Based on these considerations, we adopt explicit rejection in Zemlyanika.

**Revisiting decapsulation failures.** As mentioned earlier, decapsulation failures represent a significant drawback of lattice-based KEMs. Each failure may potentially leak partial information about the secret. To prevent them, KEM parameters are typically chosen so that the probability of a decapsulation failure, denoted by $\delta$, is negligible – often targeted to be much smaller than $2^{-100}$. In all modern KEMs, the requirements for $\delta$ are typically derived from the security bounds established in [26]. However, as noted

in [28], these bounds appear to be conservative. Based on the refined analysis in [28], we revisit the requirements for $\delta$, which allows us to improve both performance and security to attacks on the underlying M-LWE problem. At this stage, we apply the revised requirements only to a single parameter set.

**Results.** Thanks to the design choices described above, Zemlyanika offers several notable advantages. According to performance benchmarks (see Table 4), the `C` implementation of Zemlyanika outperforms the reference `C` implementations of both Kyber and Saber. Zemlyanika features shorter public keys than Kyber. The ciphertext size matches that of Kyber for two parameter sets. Zemlyanika achieves the same level of security as Kyber and Saber. Overall, these results position Zemlyanika as a competitive alternative to the current state-of-the-art lattice-based KEMs.

### 1.2 Acknowledgments

The authors would like to express their deep gratitude to Mikhail Kudinov for the insightful discussions on the security proofs of the Fujisaki-Okamoto transformation. We would also like to thank Denis Nabokov for the valuable discussions on the attacks on the LWE-based schemes. Additionally, we extend our thanks to all members of the R&D department at QApp[1] for their helpful discussions during the development of the scheme.

## 2 Preliminaries

### 2.1 Notations

We denote the ring of integers modulo $q$ by $\mathbb{Z}_q$. For any integer $a$, there exists a unique element $r \in \mathbb{Z}_q$ such that $0 \leq r < q$ and $a = k \cdot q + r$ for some $k \in \mathbb{Z}$. This element $r$ is obtained by reducing $a$ modulo $q$, which we denote as $r = a \bmod q$. We define the polynomial ring $\mathbb{Z}[x]/(x^n + 1)$, where $n$ is a power of two, and denote it by $R$. The polynomial $x^n + 1$ in this case is the $2n$-th cyclotomic polynomial. We also define the quotient ring $R_q = R/(q) = \mathbb{Z}_q[x]/(x^n + 1)$.

Elements of $R_q$, $R$, $\mathbb{Z}_q$, or $\mathbb{Z}$ are denoted by lowercase letters in regular font, such as $a$, $b$, $c$. The distinction between a scalar from $\mathbb{Z}_q$ (or $\mathbb{Z}$) and a polynomial from $R_q$ (or $R$) will be clear from the context. Vectors over $R_q$ (or $R$), as well as vectors over $\mathbb{Z}_q$ (or $\mathbb{Z}$), are denoted by bold lowercase letters, such as $\boldsymbol{a}$, $\boldsymbol{b}$, $\boldsymbol{c}$. By default, all vectors are columns. Matrices with entries in $R_q$ or $\mathbb{Z}_q$ are denoted by bold uppercase letters, such as $\boldsymbol{A}$, $\boldsymbol{B}$, $\boldsymbol{C}$.

[1] `https://qapp.tech`

For a vector $\boldsymbol{a}$ (or a matrix $\boldsymbol{A}$), the transpose is denoted by $\boldsymbol{a}^T$ (respectively, $\boldsymbol{A}^T$).

The notation $a \leftarrow \chi$ indicates that the element $a$ is sampled according to the distribution $\chi$. If $\boldsymbol{a}$ is a vector of length $k$, then $\boldsymbol{a} \leftarrow \chi^k$ denotes that each entry of $\boldsymbol{a}$ is sampled independently according to $\chi$. Let $S$ be a finite set. The notation $a \leftarrow S$ means that $a$ is chosen uniformly at random from $S$.

If the algorithm Alg is deterministic, then the expression $b := \mathrm{Alg}(a)$ denotes that the output of Alg on input $a$ is $b$. When Alg is a probabilistic algorithm, we write $b \leftarrow \mathrm{Alg}(a)$ to indicate that $b$ is obtained by executing of Alg on input $a$. In cases where it is necessary to make the randomness $r$ of a probabilistic algorithm explicit, we write $b := \mathrm{Alg}(a; r)$ instead of $b \leftarrow \mathrm{Alg}(a)$. When the randomness $r$ is provided explicitly, the algorithm Alg becomes deterministic with respect to $r$. Let $B$ be a boolean statement. We write $b = [\![B]\!]$ to denote the bit that is equal to 1 if $B$ is true and 0 otherwise.

We adopt the code-based game framework introduced in [43, 11]. A game is a program consisting of a sequence of procedures, some of which are associated with an adversary. We say that an adversary $\mathcal{A}$ wins a game $G$ if it terminates with output 1. This event is denoted as $G^{\mathcal{A}} \Rightarrow 1$. Since the game's output is a random variable, we write $\Pr[G^{\mathcal{A}} \Rightarrow 1]$ to denote the probability that $\mathcal{A}$ wins $G$. To indicate that an adversary $\mathcal{A}$ has access to an oracle $\mathcal{O}$, we write $\mathcal{A}^{\mathcal{O}(\cdot)}$.

## 2.2 Lattices and Hard Problems on Lattices

Let $E$ be a Euclidean space. A subset $\Lambda \subseteq E$ is called a *lattice* iff there exist vectors $\boldsymbol{b}_1, \boldsymbol{b}_2, \ldots, \boldsymbol{b}_m \in E$ such that

$$\Lambda = \left\{ \sum_{i=1}^{m} c_i \boldsymbol{b}_i \;\middle|\; c_i \in \mathbb{Z} \right\}.$$

If $\Lambda$ cannot be generated by fewer than $m$ vectors, then $m$ is called the *rank* of $\Lambda$, denoted by $\mathrm{rk}(\Lambda)$. The *dimension* of $\Lambda$ is defined as the dimension of the space $E$. The vectors $\boldsymbol{b}_1, \boldsymbol{b}_2, \ldots, \boldsymbol{b}_m$ form a *basis* of the lattice $\Lambda$. A basis is often represented by a matrix $\boldsymbol{B}$ whose columns are the basis vectors $\boldsymbol{b}_i$. Note that a lattice has infinitely many different bases.

Many geometric problems related to lattices are believed to be computationally hard. The *Shortest Vector Problem* (SVP) is the canonical example of such a problem. Given a basis $\boldsymbol{B}$ of a lattice $\Lambda$, the SVP requires to find a

nonzero lattice vector of minimal length. This problem is known to be NP-hard under randomized reductions [1]. The length of the shortest nonzero vector in $\Lambda$ is denoted by $\lambda_1(\Lambda)$.

In cryptographic applications, the goal is often to find short nonzero vectors in a lattice $\Lambda$, whose length is at most $\lambda_1(\Lambda)$ scaled by $\mathrm{poly}(n)$, where $n = \mathrm{rk}(\Lambda)$. For this reason, the approximate version of the Shortest Vector Problem, denoted by $\mathrm{SVP}_\gamma$, is considered. Given a basis $\boldsymbol{B}$ of a lattice $\Lambda$ and an approximation factor $\gamma(n)$ depending on $n = \mathrm{rk}(\Lambda)$, the problem $\mathrm{SVP}_\gamma$ asks for a vector $\boldsymbol{x} \in \Lambda \setminus \{\mathbf{0}\}$ such that $||\boldsymbol{x}|| \leq \gamma(n) \cdot \lambda_1(\Lambda)$. It is well known that $\mathrm{SVP}_\gamma$ can be solved efficiently when $\gamma(n)$ is exponential. However, no efficient algorithm is known for solving $\mathrm{SVP}_\gamma$ with polynomial approximation factors.

Moreover, it is not only hard to find the shortest vector in a lattice but also to estimate its length. The $\mathrm{GapSVP}_\gamma$ problem asks to approximate $\lambda_1(\Lambda)$ for a given lattice $\Lambda$. More precisely, given a basis $\boldsymbol{B}$ of the lattice $\Lambda$, a real number $r > 0$, and an approximation factor $\gamma(n)$ depending on $n = \mathrm{rk}(\Lambda)$, the $\mathrm{GapSVP}_\gamma$ problem requires to determine whether $\lambda_1(\Lambda) \leq r$ or $\lambda_1(\Lambda) > \gamma(n) \cdot r$. If $r < \lambda_1(\Lambda) \leq \gamma(n) \cdot r$, any answer is considered correct. The problem $\mathrm{GapSVP}_\gamma$ is NP-hard under randomized reductions for $\gamma < \sqrt{2}$ [1]. At the same time, it is known that $\mathrm{GapSVP}_{\sqrt{n}} \in \mathrm{NP} \cap \text{co-NP}$ [35], which implies that $\mathrm{GapSVP}_\gamma$ is unlikely to be NP-hard for $\gamma(n) = \omega(\sqrt{n})$, unless NP = co-NP. Nevertheless, no efficient algorithm is currently known for solving $\mathrm{GapSVP}_\gamma$ with $\gamma(n) = \mathrm{poly}(n)$.

Another hard geometric problem on lattices is the Shortest Independent Vectors Problem (SIVP). To define it, let us introduce the notion of successive minima. The $i$-th successive minimum $\lambda_i(\Lambda)$ of a lattice $\Lambda$ is the smallest radius $r$ such that the open ball of radius $r$ centered at the origin contains $i$ linearly independent vectors from $\Lambda$. Clearly, the length of the shortest nonzero vector of a lattice is the first successive minimum of this lattice. The SIVP asks to find $n$ linearly independent vectors $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n \in \Lambda$ such that $\max_i \|\boldsymbol{v}_i\| = \lambda_n(\Lambda)$. The approximate version of SIVP is defined analogously to the approximate SVP. To date, no polynomial-time algorithm is known for solving approximate SIVP with polynomial approximation factors.

## 2.3 Module-LWE

The Module-LWE (M-LWE) problem is a special case of the Learning With Errors (LWE) problem originally proposed by O. Regev [41]. The LWE problem consists in distinguishing uniform samples $(\boldsymbol{a}_i, b_i) \leftarrow \mathbb{Z}_q^n \times \mathbb{Z}_q$ from samples of the form $(\boldsymbol{a}_i, b_i)$, where $\boldsymbol{a}_i \leftarrow \mathbb{Z}_q^n$ and $b_i = \langle \boldsymbol{a}_i, \boldsymbol{s} \rangle + e_i \bmod q$, for a secret vector $\boldsymbol{s} \leftarrow \chi_s^n$ and noise term $e_i \leftarrow \chi_e$. The distributions $\chi_s$ and $\chi_e$ are

typically discrete Gaussians with mean zero and small standard deviation. In practice, the number of such samples $(\boldsymbol{a}_i, b_i)$ is bounded by some integer $m$. Hence, the LWE problem can be reformulated as distinguishing the vector $\boldsymbol{b} = \boldsymbol{A} \cdot \boldsymbol{s} + \boldsymbol{e} \bmod q$ from a uniformly random vector in $\mathbb{Z}_q^m$. LWE can be also viewed as solving a system of noisy linear equations, where $\boldsymbol{s}$ is the vector of $n$ unknowns. The main hardness result from [41] states that if there exists an efficient algorithm that solves LWE with $n$ variables, then there exists a quantum algorithm that can solve both $\text{SIVP}_\gamma$ and $\text{GapSVP}_\gamma$ with polynomial approximation factor $\gamma(n)$ on **any** $n$-dimensional lattice.

In cryptographic applications, a structured variant of LWE is typically employed to improve efficiency. In these structured variants, the matrix $\boldsymbol{A}$ has a specific algebraic form that enables more compact key and ciphertext representations, as well as faster arithmetic.

M-LWE is one such structured variant. It replaces vectors over $\mathbb{Z}_q$ with vectors over the ring $R_q$. The goal is to distinguish between uniform samples $(\boldsymbol{a}_i, b_i) \leftarrow R_q^k \times R_q$ and samples of the form $(\boldsymbol{a}_i, b_i)$, where $\boldsymbol{a}_i \leftarrow R_q^k$ and

$$b_i = \langle \boldsymbol{a}_i, \boldsymbol{s} \rangle + e_i,$$

with secret $\boldsymbol{s} \leftarrow \chi_s^{n \cdot k}$ and noise polynomial $e_i \leftarrow \chi_e^n$. The distributions $\chi_s$ and $\chi_e$ are usually the same as those used in LWE. Here, each component of the vectors $\boldsymbol{a}_i$ and $\boldsymbol{s}$ is a polynomial in $R_q$, and the inner product $\langle \boldsymbol{a}, \boldsymbol{s} \rangle$ is defined as $\sum_{i=1}^{k} a_i \cdot s_i$, where the multiplication $a_i \cdot s_i$ is carried out modulo $(x^n + 1, q)$. Let $a, s \in R_q$, where $a(x) = \sum_{j=0}^{n-1} a_j x^j$ and $s(x) = \sum_{j=0}^{n-1} s_j x^j$. Then their product $a \cdot s$ in $R_q$ can be represented in matrix form as:

$$a \cdot s = \begin{pmatrix} a_0 & -a_{n-1} & \cdots & -a_1 \\ a_1 & a_0 & \cdots & -a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & a_{n-2} & \cdots & a_0 \end{pmatrix} \cdot \begin{pmatrix} s_0 \\ s_1 \\ \vdots \\ s_{n-1} \end{pmatrix}$$

We denote the matrix above by $\text{rot}(a)$. The matrix $\boldsymbol{A}$ in the M-LWE problem has the following structure:

$$\boldsymbol{A} = \begin{pmatrix} \text{rot}(a_{1,1}) & \text{rot}(a_{1,2}) & \dots & \text{rot}(a_{1,k}) \\ \text{rot}(a_{2,1}) & \text{rot}(a_{2,2}) & \dots & \text{rot}(a_{2,k}) \\ \vdots & \vdots & \ddots & \vdots \\ \text{rot}(a_{m,1}) & \text{rot}(a_{m,2}) & \dots & \text{rot}(a_{m,k}) \end{pmatrix},$$

where $m$ is the number of M-LWE samples, and $a_{i,j}$ denotes the $j$-th coordinate of the vector $\boldsymbol{a}_i$. Thus, the M-LWE problem can be viewed as a classical

LWE instance with $n \cdot k$ variables and $n \cdot m$ samples. However, this additional structure theoretically reduces the hardness of the problem. In [32], the authors showed that an efficient algorithm solving M-LWE over the module $R_q^k$ would imply an efficient algorithm for solving $\text{SIVP}_\gamma$ with polynomial $\gamma(n)$ on **any** module lattice in $(\mathbb{Q}[x]/(x^n+1))^k$. In other words, the hardness of the M-LWE problem is based on the complexity of $\text{SIVP}_\gamma$ on a special class of lattices known as module lattices. At present, no known algorithms exploit the additional module structure to solve M-LWE more efficiently than generic LWE.

Let $q, k, m$ be positive integers, $n$ a power-of-two, and let $\chi_s$ and $\chi_e$ be probability distributions over $\mathbb{Z}$. The M-LWE game with adversary $\mathcal{A}$ is defined in Figure 1.

**GAME** M-LWE

$\boldsymbol{A} \leftarrow R_q^{m \times k}$
$\boldsymbol{s} \leftarrow \chi_s^{n \times k}$
$\boldsymbol{e} \leftarrow \chi_e^{n \times m}$
coin $\leftarrow \{0,1\}$
**if** coin $= 0$ **then**
    $\boldsymbol{b} := \boldsymbol{A} \cdot \boldsymbol{s} + \boldsymbol{e}$
**else**
    $\boldsymbol{b} \leftarrow R_q^m$
**end if**
coin' $\leftarrow \mathcal{A}(\boldsymbol{A}, \boldsymbol{b})$
**return** $[\![$coin' $=$ coin$]\!]$

Figure 1: M-LWE game

The advantage of an adversary $\mathcal{A}$ in the M-LWE game with parameters $n, q, k, \chi_s, \chi_e$ and $m$ is defined as

$$\text{Adv}^{\text{M-LWE}}_{n,q,k,\chi_s,\chi_e,m}(\mathcal{A}) = 2\left|\Pr[\text{M-LWE}^{\mathcal{A}} \Rightarrow 1] - \frac{1}{2}\right|.$$

## 2.4 Cryptographic Definitions

A public-key encryption (PKE) scheme consists of three polynomial-time algorithms defined with respect to a finite message space $\mathcal{M}$.

- The probabilistic key generation algorithm KeyGen outputs a pair consisting of a public key and a secret key: $(pk, sk) \leftarrow \text{KeyGen}()$.

- The probabilistic encryption algorithm Enc takes as input a public key $pk$ and a message $m \in \mathcal{M}$, and outputs a ciphertext: $ct \leftarrow \text{Enc}(pk, m)$. If we wish to make the randomness $r$ explicit, we write: $ct := \text{Enc}(pk, m; r)$, following [26, 27].

- The deterministic decryption algorithm Dec takes a secret key $sk$ and a ciphertext $ct$ as input and outputs either a message $m \in \mathcal{M}$ or a special symbol $\perp \notin \mathcal{M}$ indicating rejection: $m := \text{Dec}(sk, ct)$.

We say that a **decryption failure** occurs if decryption does not return the original message, that is,

$$\text{Dec}(sk, ct) \neq m,$$

for some ciphertext $ct \leftarrow \text{Enc}(pk, m)$, message $m \in \mathcal{M}$, and key pair $(pk, sk) \leftarrow \text{KeyGen}()$.

Following [26, 27], we define the standard security notion for public-key encryption: *Indistinguishability under Chosen-Plaintext Attack (IND-CPA)*. The IND-CPA security game with an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ is defined in Figure 2. The *advantage* of the adversary $\mathcal{A}$ in this game is defined as:

$$\text{Adv}^{\text{IND-CPA}}_{\text{PKE}}(\mathcal{A}) = 2 \cdot \left| \Pr[\text{IND-CPA}^{\mathcal{A}} \Rightarrow 1] - \frac{1}{2} \right|.$$

**GAME** IND-CPA

1: $(pk, sk) \leftarrow \text{KeyGen}()$
2: $b \leftarrow \{0, 1\}$
3: $(m_0, m_1, st) \leftarrow \mathcal{A}_1(pk)$
4: $c^* \leftarrow \text{Enc}(pk, m_b)$
5: $b' \leftarrow \mathcal{A}_2(pk, c^*, st)$
6: **return** $[\![b' = b]\!]$

Figure 2: IND-CPA game for PKE

A *key encapsulation mechanism* (KEM) scheme consists of three polynomial-time algorithms and is associated with a key space $\mathcal{K}$. A KEM enables a sender to securely transmit a cryptographic key using the receiver's public key.

- The probabilistic key generation algorithm KeyGen outputs a public/secret key pair: $(pk, sk) \leftarrow \text{KeyGen}()$.

- The probabilistic encapsulation algorithm Encaps takes a public key $pk$ as input and outputs a key-ciphertext pair $(K, ct) \leftarrow \text{Encaps}(pk)$, where $K \in \mathcal{K}$ is the shared key and $ct$ is the corresponding ciphertext.
- The deterministic decapsulation algorithm Decaps takes as input a secret key $sk$ and a ciphertext $ct$, and outputs either a key $K$ or a special symbol $\perp \notin \mathcal{K}$ to indicate rejection.

The decapsulation algorithm enables verification of the derived key $K$ for correctness. A **decapsulation failure** occurs when $\text{Decaps}(sk, ct) \neq K$ for a ciphertext $ct$ generated as part of $(K, ct) \leftarrow \text{Encaps}(pk)$, with $(pk, sk) \leftarrow \text{KeyGen}()$. Following [27], we say that a KEM is $\delta$-*correct* if:

$$\Pr[\text{Decaps}(sk, ct) \neq K] \leq \delta,$$

where the probability is taken over the randomness of key generation and encapsulation.

Following [26, 27], we adopt the standard security notion for key encapsulation known as *indistinguishability under chosen-ciphertext attack* (IND-CCA). The IND-CCA security game with adversary $\mathcal{A}$ is formalized in Figure 3. The advantage of the adversary $\mathcal{A}$ is defined as:

$$\text{Adv}^{\text{IND-CCA}}_{\text{KEM}}(\mathcal{A}) = 2\left|\Pr\left[\text{IND-CCA}^{\mathcal{A}} \Rightarrow 1\right] - \frac{1}{2}\right|.$$

**GAME** IND-CCA

1: $(pk, sk) \leftarrow \text{KeyGen}$
2: $b \leftarrow \{0, 1\}$
3: $(K_0^*, c^*) \leftarrow \text{Encaps}(pk)$
4: $K_1^* \leftarrow \mathcal{K}$
5: $b' \leftarrow \mathcal{A}^{\text{DEC}(\cdot)}(pk, c^*, K_b^*)$
6: **return** $[\![b' = b]\!]$

**DEC**$(c \neq c^*)$

1: $K := \text{Decaps}(sk, c)$
2: **return** $K$

Figure 3: Game IND-CCA for KEM

We also define some symmetric primitives used in our scheme. The first is an *extendable output function* (XOF), which can be viewed as a generalization of a cryptographic hash function. Unlike a standard hash function, a XOF accepts an additional parameter specifying the desired output length [36]. Well-known examples of XOFs include SHAKE128 and SHAKE256, both

based on the Keccak sponge construction [13]. In our work, we denote the XOF as a function $\mathcal{X} : \{0,1\}^* \to \{0,1\}^*$, where the output length is determined by the context.

The second symmetric primitive is a *pseudorandom function* (PRF). A PRF is a keyed function whose output is computationally indistinguishable from random, provided the key remains secret. Let $\mathcal{K}$ denote the key space, $n, m > 0$ be integers, $\mathcal{L} = \emptyset$ and a function $F : \mathcal{K} \times \{0,1\}^n \to \{0,1\}^m$ be a PRF. Following the notation introduced above and the definitions in [36, 44], the standard security notion for PRFs is defined in Figure 4. The advantage of an adversary $\mathcal{A}$ in distinguishing $F$ from a truly random function is given by

$$\mathrm{Adv}_F^{\mathrm{PRF}}(\mathcal{A}) = 2\left|\Pr\left[\mathrm{PRF}^{\mathcal{A}} \Rightarrow 1\right] - \frac{1}{2}\right|.$$

In our scheme, the function $\mathcal{P} : \{0,1\}^{256} \times \{0,1\}^* \to \{0,1\}^*$ acts as a PRF. The output length of $\mathcal{P}$ is determined by context.

**GAME** PRF

$k \leftarrow \mathcal{K}$
$b \leftarrow \{0,1\}$
$b' \leftarrow \mathcal{A}^{\mathcal{O}_{F_k}(\cdot)}()$
**return** $[\![b' = b]\!]$

$\mathcal{O}_{F_k}(x \in \{0,1\}^n)$

**if** $b = 0$ **then**
    **if** $(x, y') \notin \mathcal{L}$ **then**
        $y \leftarrow \{0,1\}^m$
        $\mathcal{L} := \mathcal{L} \cup (x, y)$
    **else**
        $y := y'$
    **end if**
**else**
    $y \leftarrow F(k, x)$
**end if**
**return** $y$

Figure 4: Game PRF

## 2.5 Fujisaki–Okamoto Transform

The Fujisaki–Okamoto (FO) transform was originally proposed in [24] to construct an IND-CCA secure public-key encryption (PKE) scheme in the Random Oracle Model (ROM). It was later revisited in [26], where FO-like transforms were introduced and analyzed for converting passively secure encryption scheme into actively secure KEM.

In our scheme, we use the transform $\mathrm{FO}_m^{\perp}$, which consists of two parts: T and $\mathrm{U}_m^{\perp}$. The first part, T, transforms a probabilistic encryption scheme $\mathrm{PKE} = (\mathrm{KeyGen}, \mathrm{Enc}, \mathrm{Dec})$ into a deterministic encryption scheme $\mathrm{PKE}^G = (\mathrm{KeyGen}, \mathrm{Enc}^G, \mathrm{Dec}^G)$ using a hash function $G$. The encryption algorithm $\mathrm{Enc}^G$ is the "derandomized" Enc:

$$\mathrm{Enc}^G(pk, m) := \mathrm{Enc}(pk, m; G(m)).$$

The decryption algorithm $\mathrm{Dec}^G$ first computes $m' := \mathrm{Dec}(sk, ct)$ and then checks whether $ct = \mathrm{Enc}^G(pk, m')$. If the check passes, it outputs $m'$. Otherwise, it returns the failure symbol $\perp$. The second part, $\mathrm{U}_m^{\perp}$, constructs a KEM scheme $\mathrm{KEM}_m^{\perp}$ from $\mathrm{PKE}^G$ using another hash function $H$. The encapsulation algorithm is defined as:

$$(K := H(m),\ ct := \mathrm{Enc}^G(pk, m)) \leftarrow \mathrm{Encaps}(pk),$$

where $m$ is chosen uniformly at random from the message space $\mathcal{M}$. The decapsulation algorithm works as follows: it computes $m := \mathrm{Dec}^G(sk, ct)$. If $m \neq \perp$, it returns $K := H(m)$. Otherwise, it returns $\perp$. We note that the variant $\mathrm{FO}_{m,c}^{\perp}$, where the key is computed as $K := H(m, c)$, is not considered here, since the simpler transform $\mathrm{FO}_m^{\perp}$ provides equivalent security guarantees (see details in [14]).

It is common to say that $\mathrm{FO}_m^{\perp}$ employs an *explicit rejection*, as it clearly indicates a decapsulation failure. In contrast, the transformation $\mathrm{FO}_m^{\not\perp}$ uses an *implicit rejection*, where the failure of the decapsulation algorithm is concealed by outputting a random key. Key encapsulation mechanisms (KEMs) submitted to the NIST competition, such as Kyber and Saber, adopt implicit rejection, as a security proof for $\mathrm{FO}_m^{\not\perp}$ is available in the QROM. Recently, a security proof for $\mathrm{FO}_m^{\perp}$ in the $\mathrm{eQROM}_{\mathrm{Enc}}$ model was proposed in [28]. Informally, $\mathrm{eQROM}_{\mathrm{Enc}}$ provides an adversary with greater capabilities than the standard QROM (see [28] for details). Moreover, work [29] established the equivalence between the explicit and implicit rejection variants within the $\mathrm{eQROM}_{\mathrm{Enc}}$ model.

## 2.6 Decapsulation Failures

Decapsulation failures play a critical role in the security analysis of KEMs. Each failure may potentially leak secret information to an adversary, as its occurrence (for a fixed ciphertext) depends only on the secret vectors $\boldsymbol{s}$ and $\boldsymbol{e}$. In other words, a decapsulation failure for some $ct \in \mathcal{C}$ may enable an adversary to guess some coefficients of the secret vectors. The amount of information leaked through such failures is analyzed in [23]. An adversary

could exploit this by submitting a large number of ciphertexts to an honest party, some of which will trigger decapsulation failures. The resulting information can then be used to recover parts of the secret key. For this reason, the probability of decapsulation failure must be negligible for any key pair $(pk, sk) \leftarrow \text{KeyGen}()$.

The impact of decapsulation failures on the security of KEMs constructed via the FO-type transformation is analyzed in [26]. According to this work, the impact is directly related to the so-called $\delta$-correctness of the underlying public-key encryption (PKE) scheme. A PKE scheme is said to be $\delta$-correct if

$$\mathrm{E}\left[\max_{m \in \mathcal{M}} \Pr_{c \leftarrow \text{Enc}(pk,m)}[\text{Dec}(sk, c) \neq m]\right] \leq \delta,$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}()$. If $\delta \neq 0$, then the advantage of an IND-CCA adversary against the KEM increases by the adversary's advantage in the COR-RO game against $\text{PKE}^G$. In the COR-RO game (defined in [26]), the adversary is given both the public and secret keys and has access to an oracle $G$. The goal of the adversary is to find a message that causes a decryption failure. If the underlying $\text{PKE}^G$ scheme is $\delta_1$-correct, the adversary's advantage in the COR-RO game is upper bounded by $\delta_1$. As shown in [26], for a $\delta$-correct PKE scheme, we have $\delta_1 = q_G \cdot \delta$ in the random oracle model (ROM), where $q_G$ is the number of queries to the oracle $G$. In the quantum random oracle model (QROM), the bound becomes $\delta_1 \leq 8q_G^2 \cdot \delta$.

Many lattice-based schemes, including Kyber and Saber, employ the described approach to estimate a suitable value of $\delta$. It is worth noting that, according to the results of [26], achieving $\lambda$-bit security requires $\delta \leq 2^{-\lambda}$. However, Kyber and Saber do not satisfy this condition for their third and fifth parameter sets.

Following [28], we argue that the estimation of the impact of decapsulation failures given in [26] is conservative. The primary reason is that the COR-RO game provides the adversary access to the secret key, which is an unrealistic assumption. With access to the secret key, the adversary can locally verify the correctness of any ciphertext. In practice, however, an adversary can only test ciphertext correctness by sending it to an honest party who possesses the secret key. This distinction has two important implications. First, it rules out the possibility of using Grover's search to efficiently find incorrect ciphertexts. Second, the number of queries to the decapsulation or decryption oracle is typically limited in real-world scenarios [28], which reduces the adversary's power. The efficiency of existing attacks that exploit decapsulation failures [23, 22] heavily depends on the number of decryption queries allowed. For instance, the attack proposed in [23] against

Saber with the third parameter set requires approximately $2^{139}$ operations, in contrast to the $2^{184}$ operations needed for the best known attack on the underlying M-LWR problem. However, this attack also requires around $2^{131}$ decryption queries, which far exceeds any practical limit.

An alternative approach for estimating the impact of decapsulation failures on the security of a KEM was proposed in [28]. The core idea is to replace the COR-RO game with the FFP-ATK game, where $\mathrm{ATK} \in \{\mathrm{CPA}, \mathrm{CCA}\}$. In this setting, the adversary is given access to an oracle $O_{\mathrm{ATK}}$ defined as follows:

$$O_{\mathrm{ATK}} = \begin{cases} -, & \text{if ATK} = \mathrm{CPA}, \\ \mathrm{DEC}(\cdot), & \text{if ATK} = \mathrm{CCA}. \end{cases}$$

A detailed description of the FFP-ATK game for $\mathrm{PKE}^G$ is provided in Figure 5. The advantage of an adversary $\mathcal{A}$ in the FFP-ATK game is defined as

$$\mathrm{Adv}^{\text{FFP-ATK}}_{\mathrm{PKE}^G}(\mathcal{A}) = \Pr\left[\text{FFP-ATK}^{\mathcal{A}}_{\mathrm{PKE}^G} \Rightarrow 1\right].$$

**GAME** FFP-ATK

1: $(pk, sk) \leftarrow \mathrm{KeyGen}$
2: $m \leftarrow \mathcal{A}^{O_{\mathrm{ATK}}, G}(pk)$
3: $c := \mathrm{Enc}^G(pk, m)$
4: $m' := \mathrm{Dec}^G(sk, c)$
5: **return** $[\![m' \neq m]\!]$

**DEC**$(c)$

1: $m := \mathrm{Dec}^G(sk, c)$
2: **return** $m$

Figure 5: Game FFP-ATK for $\mathrm{PKE}^G$

We recall the definition of a $\gamma$-spread PKE scheme. A public-key encryption scheme is said to be $\gamma$-spread if, for all key pairs $(pk, sk)$ and all messages $m \in \mathcal{M}$, it holds that

$$\max_{c \in \mathcal{C}} \Pr[\mathrm{Enc}(pk, m) = c] \leq 2^{-\gamma}.$$

According to [28], for a $\gamma$-spread $\mathrm{PKE}^G$ scheme in the ROM, the following bound holds:

$$\mathrm{Adv}^{\text{FFP-CCA}}_{\mathrm{PKE}^G}(\mathcal{A}) \leq (q_D + 1) \cdot \mathrm{Adv}^{\text{FFP-CPA}}_{\mathrm{PKE}^G}(\mathcal{B}) + q_D \cdot 2^{-\gamma}, \tag{1}$$

where $q_D$ is the number of decryption queries. The adversary $\mathcal{B}$ makes at most the same number of queries to $G$ as $\mathcal{A}$ and runs in approximately the same

time. An analogous bound also holds in the quantum setting. Specifically, for a $\gamma$-spread $\mathrm{PKE}^G$ scheme in the $\mathrm{eQROM}_{\mathrm{Enc}}$ model, we have [28]:

$$\mathrm{Adv}^{\text{FFP-CCA}}_{\mathrm{PKE}^G}(\mathcal{A}) \leq (q_D+1)\cdot \mathrm{Adv}^{\text{FFP-CPA}}_{\mathrm{PKE}^G}(\mathcal{B}) + 12q_D(q_G+4q_D)\cdot 2^{-\gamma/2}, \quad (2)$$

where $q_D$ is the number of decryption queries, $q_G$ is the number of queries to $G$, and the running time of $\mathcal{B}$ satisfies $\mathrm{Time}(\mathcal{B}) = \mathrm{Time}(\mathcal{A}) + O(q_D)$.

Several bounds are known for estimating $\mathrm{Adv}^{\text{FFP-CPA}}_{\mathrm{PKE}^G}(\mathcal{A})$. First, in the ROM, we can use the fact that

$$\mathrm{Adv}^{\text{FFP-CCA}}_{\mathrm{PKE}^G}(\mathcal{A}) \leq \mathrm{Adv}^{\text{COR-RO}}_{\mathrm{PKE}^G}(\mathcal{A}) = q_G\cdot\delta, \quad (3)$$

which directly follows from the relationship between the FFP-CPA and COR-RO games. In the $\mathrm{eQROM}_{\mathrm{Enc}}$, the advantage in the FFP-CPA game is estimated in [30] as follows:

$$\mathrm{Adv}^{\text{FFP-CCA}}_{\mathrm{PKE}^G}(\mathcal{A}) \leq 10(q_G+q_D+1)^2\cdot\delta. \quad (4)$$

However, we note that these bounds are closely related or equivalent to those presented in earlier works [26, 27]. As a result, we consider them to be conservative, and in some cases, we prefer to work with the quantity $\mathrm{Adv}^{\text{FFP-CPA}}_{\mathrm{PKE}^G}(\mathcal{B})$ explicitly.

## 2.7 Centered Binomial Distribution

A centered binomial distribution $B_\eta$ with parameter $\eta > 0$ is a probability distribution over the set $\{-\eta, \ldots, 0, \ldots, \eta\}$, defined by

$$\Pr_{x\leftarrow B_\eta}[x=t] = \frac{\binom{2\eta}{\eta+t}}{2^{2\eta}}.$$

To sample from $B_\eta$, it suffices to generate a bit string

$$(a_0, a_1, \ldots, a_{2\eta-1}) \leftarrow \{0,1\}^{2\eta}$$

and output

$$\sum_{i=0}^{\eta-1}(a_{2i} - a_{2i+1}).$$

## 2.8 Compression and Decompression

Both operations are used in the proposed KEM. Compression is applied to the ciphertext at the end of the encapsulation algorithm. It reduces the ciphertext size at the cost of partial information loss. Decompression is applied

to the compressed ciphertext to partially recover the original data. These operations were first introduced in Kyber. However, since we use a power-of-two modulus (as opposed to Kyber's prime modulus), our compression and decompression operations are slightly different.

Both operations are initially defined for integers. Let $q = 2^t$, where $t \geq 2$, and let $0 \leq d < t$ be an integer parameter. The compression operation takes an integer input $0 \leq x < q$, while the decompression operation takes an integer input $0 \leq x < 2^{t-d}$. We define both operations as follows:

$$\text{Compress}_d(x) = ((x + 2^{d-1}) \gg d) \bmod 2^{t-d};$$

$$\text{Decompress}_d(x) = x \ll d.$$

Here, $\gg$ and $\ll$ denote right and left bit shifts, respectively. When $d = 0$, both operations are considered identity functions.

**Lemma 1.** *Let $0 \leq x < q$, $0 < d < t$, and let $x' = \text{Decompress}_d(\text{Compress}_d(x))$. Let $\Delta_x = x' - x \bmod q$. Then, the inequality*

$$-2^{d-1} < \Delta_x \leq 2^{d-1}$$

*holds.*

*Proof.* Write $x = 2^d \cdot e + r$, where $0 \leq r < 2^d$ and $0 \leq e < 2^{t-d}$. Let $y = \text{Compress}_d(x)$. Then

$$y = \left\lfloor \frac{x + 2^{d-1}}{2^d} \right\rfloor \bmod 2^{t-d} = e + \left\lfloor \frac{2^{d-1} + r}{2^d} \right\rfloor \bmod 2^{t-d}.$$

**Case 1:** $r < 2^{d-1}$. Then $\left\lfloor \frac{2^{d-1}+r}{2^d} \right\rfloor = 0$, so $y = e$ and $x' = e \ll d = 2^d \cdot e$. Hence,

$$\Delta_x = x' - x \bmod q = -r \bmod q.$$

Since $0 \leq r < 2^{d-1}$, we get $-2^{d-1} < \Delta_x \leq 0$.

**Case 2:** $2^{d-1} \leq r < 2^d$. Then $\left\lfloor \frac{2^{d-1}+r}{2^d} \right\rfloor = 1$, so $y = e + 1 \bmod 2^{t-d}$.

**Subcase 2.1:** $e \neq 2^{t-d}-1$. Then $y = e+1$ and $x' = 2^d \cdot (e+1) = 2^d \cdot e + 2^d$, so

$$\Delta_x = x' - x = 2^d - r \bmod q.$$

Since $r \geq 2^{d-1}$, we get $0 < \Delta_x \leq 2^{d-1}$.

**Subcase 2.2:** $e = 2^{t-d} - 1$. Then $e + 1 \equiv 0 \bmod 2^{t-d}$, so $y = 0$ and $x' = 0$. Therefore,

$$\Delta_x = -x \bmod q = -2^d \cdot (2^{t-d} - 1) - r \bmod q = 2^d - r \bmod q,$$

and again $0 < \Delta_x \leq 2^{d-1}$.

Thus, in all cases we have

$$-2^{d-1} < \Delta_x \leq 2^{d-1}.$$

□

In other words, Lemma 1 states that the compression introduces an additional noise, and its value lies in the set $\{-2^{d-1}+1, \ldots, 2^{d-1}\}$. A nice property of $\Delta_x$ is that it is uniformly distributed when $x \leftarrow \mathbb{Z}_q$.

**Lemma 2.** *Let $t > 1$, $q = 2^t$, $0 < d < t$, and let $E$ be any element from the set $\{-2^{d-1}+1, \ldots, 2^{d-1}\}$. Then*

$$\Pr_{x \leftarrow \mathbb{Z}_q}[\Delta_x = E] = \frac{1}{2^d},$$

*where $\Delta_x$ is defined as in Lemma 1.*

*Proof.* Since $q = 2^t$ and $x \leftarrow \mathbb{Z}_q$, we can write $x = 2^d \cdot e + r$, where $e \leftarrow \mathbb{Z}_{2^{t-d}}$ and $r \leftarrow \mathbb{Z}_{2^d}$. According to the proof of Lemma 1, we have:

$$\Delta_x = \begin{cases} -r \mod q & \text{if } 0 \leq r < 2^{d-1}, \\ 2^d - r \mod q & \text{if } 2^{d-1} \leq r < 2^d. \end{cases}$$

This means that each possible value of $r$ yields a unique value of $\Delta_x$, and since $r$ is uniformly distributed over $\mathbb{Z}_{2^d}$ when $x \leftarrow \mathbb{Z}_q$, the result follows. □

When $a \in R_q$, the notation $\text{Compress}_d(a)$ (and $\text{Decompress}_d(a)$) means that the compression (or decompression) operation is applied to each coefficient of $a$. Similarly, the notation $\text{Compress}_d(\boldsymbol{a})$ (and $\text{Decompress}_d(\boldsymbol{a})$) indicates that the compression (or decompression) is applied to each polynomial in $\boldsymbol{a} \in R_q^k$.

# 3 Zemlyanika Description

Zemlyanika is an IND-CCA KEM built by applying the $\text{FO}_m^{\perp}$ transform to an IND-CPA PKE scheme. We will refer to the KEM as Zem.KEM and to the underlying public-key encryption scheme as Zem.PKE. Our PKE scheme Zem.PKE is based on the so-called LPR scheme, originally proposed in [34], which became the foundation for nearly all LWE/LWR-based public-key encryption schemes. In our construction, we use the LPR scheme in the M-LWE setting. We begin with a description of the Zem.PKE scheme.

## 3.1 Zemlyanika PKE

Recall that $R_q = \mathbb{Z}_q[x]/(x^n+1)$, where $n$ is a power of two. For all parameter sets, we fix $n = 256$. Let $q, k, \eta_s, \eta_e, d_u, d_v$ be positive integers such that $q = 2^t$ is a power of two. Let also $\mathcal{M} = R/(2) = R_2$ denote the message space. Note that a polynomial from $R_2$ can be interpreted as a binary string of length $n$. Therefore, the proposed public-key encryption scheme encrypts 32-byte messages.

First, we present the algorithm (see Algorithm 1) for generating a uniformly random matrix $\boldsymbol{A} \in R_q^{k\times k}$. To reduce the public key size, matrix $\boldsymbol{A}$ is deterministically generated from a public seed $\text{seed}_A \in \{0,1\}^{256}$. To obtain the required amount of randomness, we use an extendable output function (XOF) $\mathcal{X}$. Since $q = 2^t$, generating one random coefficient from $\mathbb{Z}_q$ requires exactly $t$ random bits. Recall that each entry of $\boldsymbol{A}$ is a polynomial in $R_q$, which consists of $n = 256$ coefficients from $\mathbb{Z}_q$. Hence, to generate the matrix $\boldsymbol{A}$, we need to sample $k^2$ random polynomials, each with 256 coefficients of $t$ bits. The total required number of bits is $\ell = 256 \cdot k^2 \cdot t$. Let $B$ be the bit string output by $\mathcal{X}(\text{seed}_A, \ell)$. We use the notation $a := \text{int}(B[i:j])$ for $j > i$, to denote that the integer $a$ is computed as $a = \sum_{b=0}^{j-i-1} B[i+b] \cdot 2^b$. Next, we

**Algorithm 1** MatrixGen($\text{seed}_A \in \{0,1\}^{256}$)

```
1:  buf := 𝒳(seed_A)
2:  b := 0
3:  for (i := 0; i < k; i := i + 1) do
4:      for (j := 0; j < k; j := j + 1) do
5:          a(x) := 0
6:          for (deg := 0; deg < 256; deg := deg + 1) do
7:              coef := int(buf[b : b + t])                ▷ t = log_2 q
8:              a(x) := a(x) + coef · x^deg
9:              b := b + t
10:         end for
11:         A[i][j] = a(x)
12:     end for
13: end for
14: return A
```

describe the algorithm for generating a polynomial in $R_q$, whose coefficients are independently sampled from the centered binomial distribution $B_\eta$ (see Algorithm 2). Recall that sampling from $B_\eta$ requires $2\eta$ pseudorandom bits per sample. Since a polynomial in $R_q$ has $n = 256$ coefficients, sampling a

polynomial from $B_\eta^{256}$ requires a total of $512\eta$ pseudorandom bits. To produce this amount of pseudorandomness, we use the pseudorandom function (PRF) $\mathcal{P}$. It takes as input a secret seed $\text{seed}_s$ (used as the key) and a nonce (used as the message). The output of $\mathcal{P}$ is then used as the source of pseudorandom bits for sampling from $B_\eta$.

**Algorithm 2** PolyFromCBD($\eta > 0$, $\text{seed}_s \in \{0,1\}^{256}$, nonce $> 0$)

buf := $\mathcal{P}(\text{seed}_s, \text{nonce})$
$s(x) = 0$
**for** $(deg := 0; deg < 256; deg := deg + 1)$ **do**
    coef := 0
    **for** $(j := 0; j < 2\eta; j := j + 2)$ **do**
        coef := coef + (buf$[2\eta \cdot deg + j]$ − buf$[2\eta \cdot deg + j + 1]$)
    **end for**
    $s(x) := s(x) + \text{coef} \cdot x^{deg}$
**end for**
**return** $s(x)$

Now, we can proceed to the description of the scheme itself. The key generation algorithm, the encryption algorithm, and the decryption algorithm are given in Algorithms 3, 4, and 5, respectively. Note that the randomness used in the encryption algorithm is provided explicitly as an input parameter.

**Decryption correctness.** First, consider the values $\boldsymbol{u}''$ and $v''$ obtained in the first two steps of the decryption algorithm. According to Lemma 2, we have:

$$\begin{cases} \boldsymbol{u}'' = \boldsymbol{u} + \boldsymbol{e}_u, \\ v'' = v + e_v, \end{cases}$$

where $\boldsymbol{e}_u \leftarrow \{-2^{d_u-1}+1, \ldots, 2^{d_u-1}\}^{256 \cdot k}$ and $e_v \leftarrow \{-2^{d_v-1}+1, \ldots, 2^{d_v-1}\}^{256}$.

Now, consider the expression for $m' = v'' - \boldsymbol{s}^T \cdot \boldsymbol{u}''$:

$$\begin{aligned} m' &= v + e_v - \boldsymbol{s}^T \cdot (\boldsymbol{u} + \boldsymbol{e}_u) \\ &= \boldsymbol{b}^T \cdot \boldsymbol{r} + e_2 + \frac{q}{2} \cdot m + e_v - \boldsymbol{s}^T \cdot (\boldsymbol{A}^T \cdot \boldsymbol{r} + \boldsymbol{e}_1 + \boldsymbol{e}_u) \\ &= (\boldsymbol{s}^T \cdot \boldsymbol{A}^T + \boldsymbol{e}^T) \cdot \boldsymbol{r} + e_2 + e_v - \boldsymbol{s}^T \cdot \boldsymbol{A}^T \cdot \boldsymbol{r} - \boldsymbol{s}^T \cdot (\boldsymbol{e}_1 + \boldsymbol{e}_u) + \frac{q}{2} \cdot m \\ &= \boldsymbol{e}^T \cdot \boldsymbol{r} - \boldsymbol{s}^T \cdot (\boldsymbol{e}_1 + \boldsymbol{e}_u) + e_2 + e_v + \frac{q}{2} \cdot m. \end{aligned}$$

Thus, the decrypted message $m'$ equals the original message $m$ iff

$$\left\| \boldsymbol{e}^T \cdot \boldsymbol{r} - \boldsymbol{s}^T \cdot (\boldsymbol{e}_1 + \boldsymbol{e}_u) + e_2 + e_v \right\|_\infty < \frac{q}{4}. \tag{5}$$

**Algorithm 3** Zem.PKE.KeyGen()

$\text{seed}_A \leftarrow \{0,1\}^{256}$
$\text{seed}_s \leftarrow \{0,1\}^{256}$
nonce $:= 0$
$\boldsymbol{A} = \text{MatrixGen}(\text{seed}_A)$
**for** $(i := 0; i < k; i := i + 1)$ **do**
    $\boldsymbol{s}[i] := \text{PolyFromCBD}(\eta_s, \text{seed}_s, \text{nonce})$
    nonce := nonce + 1
**end for**
**for** $(i := 0; i < k; i := i + 1)$ **do**
    $\boldsymbol{s}[i] := \text{PolyFromCBD}(\eta_e, \text{seed}_s, \text{nonce})$
    nonce := nonce + 1
**end for**
$\boldsymbol{b} := \boldsymbol{A} \cdot \boldsymbol{s} + \boldsymbol{e}$
$pk := (\boldsymbol{b}, \text{seed}_A)$
$sk := \boldsymbol{s}$
**return** $(pk, sk)$

We note that due to the power-of-two modulus $q$, there is some asymmetry: correct decryption is possible when some coefficients are equal to $q/4$, but fails if any coefficient equals $-q/4$. However, for simplicity, we will use inequality (5) as the correctness criterion. According to the definition of $\delta$-correctness from Section 2.6, a PKE scheme is $\delta$-correct if

$$\text{E}_{(pk,sk)\leftarrow\text{KeyGen}()}\left[\max_{m\in\mathcal{M}} \Pr_{c\leftarrow\text{Enc}(pk,m)}[\text{Dec}(sk,c) \neq m]\right] \leq \delta.$$

For Zemlyanika PKE, this can be rewritten as:

$$\Pr\left[\left\|\boldsymbol{e}^T \cdot \boldsymbol{r} - \boldsymbol{s}^T \cdot (\boldsymbol{e}_1 + \boldsymbol{e}_u) + e_2 + e_v\right\|_\infty \geq \frac{q}{4}\right] \leq \delta,$$

where $\boldsymbol{s}, \boldsymbol{e}, \boldsymbol{r}, \boldsymbol{e}_1 \leftarrow B_\eta^{256k}$, $e_2 \leftarrow B_\eta^{256}$, $\boldsymbol{e}_u \leftarrow \{-2^{d_u-1}+1, \ldots, 2^{d_u-1}\}^{256\cdot k}$, and $e_v \leftarrow \{-2^{d_v-1}+1, \ldots, 2^{d_v-1}\}^{256}$. To determine $\delta$ for a concrete parameter set, we use a Python script, following the methodology from [17, 21].

## 3.2 Zemlyanika KEM

Following [26, 27, 28], we construct our KEM based on the Zemlyanika PKE scheme using the transform $\text{FO}_m^\perp$. Let $H : \{0,1\}^* \to \{0,1\}^{256}$ and $G : \{0,1\}^* \to \{0,1\}^{512}$ be cryptographic hash functions. The key generation

**Algorithm 4** Zem.PKE.Enc($pk = (\boldsymbol{b}, \text{seed}_A), m \in \mathcal{M}; \text{rnd} \in \{0,1\}^{256}$)

nonce := 0
$\boldsymbol{A}$ = MatrixGen($\text{seed}_A$)
**for** ($i := 0; i < k; i := i + 1$) **do**
    $\boldsymbol{r}[i]$ := PolyFromCBD($\eta_s$, rnd, nonce)
    nonce := nonce + 1
**end for**
**for** ($i := 0; i < k; i := i + 1$) **do**
    $\boldsymbol{e_1}[i]$ := PolyFromCBD($\eta_e$, rnd, nonce)
    nonce := nonce + 1
**end for**
$e_2$ := PolyFromCBD($\eta_e$, rnd, nonce)
$\boldsymbol{u} := \boldsymbol{A}^T \cdot \boldsymbol{r} + \boldsymbol{e_1}$
$v := \boldsymbol{b}^T \cdot \boldsymbol{r} + e_2 + \frac{q}{2} \cdot m$
$\boldsymbol{u'}$ := $\text{Compress}_{d_u}(\boldsymbol{u})$
$v'$ := $\text{Compress}_{d_v}(v)$
**return** $ct := (\boldsymbol{u'}, v')$

algorithm, the encapsulation algorithm, and the decapsulation algorithm are presented in Algorithms 6, 7, and 8, respectively.

**Decapsulation failures:** It is clear that a decapsulation failure occurs if and only if a decryption failure has taken place.

## 3.3 Parameter Sets

We propose four parameter sets: `Z512`, `Z768-R`, `Z768-C`, and `Z1024` (see Table 1), providing three security levels roughly corresponding to the NIST security levels 1, 3, and 5. Two parameter sets, `Z768-C` and `Z768-R`, are proposed for the third security level. The difference between them lies in how the impact of decapsulation failure on security is estimated. The parameter set `Z768-C` is based on conservative requirements for $\delta$, while `Z768-R` relies on revisited (more relaxed) requirements. We recall that $n = 256$ for all parameter sets.

## 3.4 Design Rationale

**Module-LWE:** Today, the Module-LWE problem is known to achieve the best trade-off between efficiency and security among all types of structured LWE. The highly structured Ring-LWE problem [34] is currently considered unreliable for KEMs. On the other hand, known KEMs whose security is

**Algorithm 5** Zem.PKE.Dec($sk = \boldsymbol{s}, ct := (\boldsymbol{u'}, v')$)

$\boldsymbol{u''} := \text{Decompress}_{d_u}(\boldsymbol{u'})$
$v'' := \text{Decompress}_{d_v}(v')$
$m' := v'' - \boldsymbol{s}^T \cdot \boldsymbol{u''}$
**for** $(i := 0; i < 256; i := i + 1)$ **do**
    **if** $\frac{q}{4} < m'[i] \leq \frac{3q}{4}$ **then**
        $m[i] := 1$
    **else**
        $m[i] := 0$
    **end if**
**end for**
**return** $m$

**Algorithm 6** Zem.KEM.KeyGen()

$(pk, sk') \leftarrow$ Zem.PKE.KeyGen()
$sk := sk'||pk||H(pk)$
**return** $(pk, sk)$

based on the classical LWE suffer from inefficiency. The Module-LWR problem is also known to be a good choice for KEM designers. However, this problem has been studied less extensively than the Module-LWE problem. According to the status report on the third round of the NIST PQC standardization process [2], this is the main reason why NIST preferred Kyber over Saber.

**Power-of-two cyclotomic ring:** We choose the ring $R_q$ to be $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, where $(x^n + 1)$ is the $2n$-th cyclotomic polynomial. We note that all current M-LWE/M-LWR-based schemes use only cyclotomic rings due to the fact that the theoretical complexity of structured variants of LWE is much better researched for cyclotomic rings. Power-of-two cyclotomic rings are a common choice for M-LWE/M-LWR-based schemes and are used, for example, in Kyber, Saber, and NewHope. However, there also exist schemes using other cyclotomic polynomials. In Tyber, the trinomial cyclotomic polynomial $x^n - x^{n/2} + 1$ is used, where $n = 2^k \cdot 3^l$ with $k \geq 1$, $l \geq 0$. We see this choice as unnatural due to the size of the polynomials from $\mathbb{Z}_q[x]/(x^n + x^{n/2} + 1)$, which is not a power-of-two. First, the message size in bytes is no longer a power-of-two. Second, less efficient alignment can significantly reduce the overall performance. These disadvantages are common among non-power-of-two cyclotomic polynomials. Moreover, the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ has one more desirable property. Let us consider the

**Algorithm 7** Zem.KEM.Encaps($pk$)

$m \leftarrow \{0,1\}^{256}$
$(K, r) := G(m||H(pk))$
$ct := \text{Zem.PKE.Enc}(pk, m; r)$
**return** $(K, ct)$

**Algorithm 8** Zem.KEM.Decaps($sk, ct$)

$m' := \text{Zem.PKE.Dec}(sk', ct)$
$(K', r') := G(m'||H(pk))$
$ct' := \text{Zem.PKE.Enc}(pk, m'; r')$
**if** $ct' = ct$ **then**
    **return** $K'$
**else**
    **return** $\perp$
**end if**

matrix $\text{rot}(a)$ for some $a \in R_q$. This matrix determines the linear transformation $a \cdot b$, where $b \in R_q$ is the input. Any coefficient of this matrix is either a coefficient or a negation of a coefficient of the polynomial $a(x)$. On the other hand, the analogous matrix for any other cyclotomic ring is more complex; some elements are sums of polynomial coefficients. As a result, $||a \cdot b||_\infty$ tends to be smaller on average when $a, b \in R_q$. Thus, we believe that power-of-two cyclotomics yield the smallest $\delta$ among all cyclotomic rings.

**Power-of-two modulus.** We choose the modulus $q$ to be a power-of-two. In many lattice-based schemes, $q$ is selected to be a prime number in order to enable the use of the Number Theoretic Transform (NTT), which provides an efficient algorithm for multiplying high-degree polynomials. For this reason, prime moduli are used in M-LWE-based schemes such as Kyber, Dilithium, and Tyber. In contrast, M-LWR-based schemes such as Saber employ power-of-two moduli due to the simplicity of rounding operations. This choice rules out the use of NTT-based polynomial multiplication. Instead, a combination of Toom–Cook, Karatsuba, and schoolbook multiplication algorithms is typically employed. This multiplication approach is generally less efficient than NTT-based one. However, using a power-of-two modulus enables the use of built-in modular arithmetic, thereby eliminating the need for specialized reduction algorithms such as Montgomery [37] or Barrett [10] reduction. The impact of modular reduction on overall performance can be significant. For example, an optimization aimed at reducing the number of modular reductions in Kyber was proposed in [45]. The authors demonstrated that their approach significantly improves performance: the decap-

Table 1: Zemlyanika Parameter Sets

| # | $q$ | $k$ | $\eta_s$ | $\eta_e$ | $(d_u, d_v)$ | $\lvert sk\rvert$, B | $\lvert pk\rvert$, B | $\lvert ct\rvert$, B | $\log_2(\delta)$ |
|---|---|---|---|---|---|---|---|---|---|
| Z512 | 1024 | 2 | 1 | 1 | (0, 6) | 832 | 672 | 768 | -149 |
| Z768-R | 1024 | 3 | 1 | 1 | (0, 0) | 1216 | 992 | 1280 | -125 |
| Z768-C | 2048 | 3 | 2 | 1 | (1, 5) | 1408 | 1088 | 1152 | -156 |
| Z1024 | 2048 | 4 | 2 | 1 | (0, 6) | 1856 | 1440 | 1568 | -168 |

sulation algorithm was accelerated by up to 1.83 times, the encapsulation by up to 1.58 times, and key generation by up to 1.41 times. Thus, built-in modular arithmetic is a notable advantage of using a power-of-two modulus. Another benefit is the efficiency of uniform sampling over $\mathbb{Z}_q$, which directly affects the performance of the scheme. In particular, both the key generation and encapsulation algorithms rely on generating random matrices with coefficients sampled uniformly from $\mathbb{Z}_q$.

**Centered binomial distribution.** Following [17, 21, 33, 6], we use the centered binomial distribution $B_\eta$ to sample the coefficients of both the secret and error vectors. The primary motivation is that constant-time sampling from the discrete Gaussian distribution is inefficient. The use of a centered binomial distribution as an alternative was first proposed in NewHope. This distribution offers two key advantages: (i) its shape closely resembles that of a discrete Gaussian, and (ii) it can be implemented efficiently in constant time. Moreover, the complexity of modern attacks on LWE-based schemes depends primarily on the standard deviations of the noise and secret distributions, rather than on their exact shapes. For these reasons, the centered binomial distribution is widely regarded as the most practical choice at present.

**Compression and decompression.** Compression of polynomial coefficients is used to reduce ciphertext size. However, this technique introduces additional noise into the ciphertext (see Lemmas 1 and 2), which in turn increases the probability of decapsulation failure. As a result, there is a trade-off between efficiency and security: the larger the compression parameter $d$, the smaller the ciphertext and the more efficient the scheme becomes – but at the cost of reduced correctness. Following Kyber, we estimate the complexity of the underlying M-LWE problem while ignoring the noise introduced by compression. This leads to conservative estimates, as the additional noise actually increases the hardness of the M-LWE problem.

**Explicit rejection.** In contrast to most modern M-LWE and M-LWR-based KEMs, we adopt explicit rejection rather than implicit rejection. This approach improves performance by eliminating one instance of randomness generation and reducing the size of the secret key. At the same time, it has

been shown in [29] that explicit rejection does not weaken security compared to implicit rejection.

**Encryption randomness depends on the public key.** This modification of the $\mathrm{FO}_m^{\perp}$ transformation is adopted following Kyber and Saber. As a result, both the session key $K$ and the ciphertext $ct$ depend on the context of the session, including the public key. The primary motivation for this design choice is to mitigate attacks that exploit precomputed messages $m$ hashed into a "weak" ciphertext. By a "weak" ciphertext, we mean one that leads to decapsulation failures more frequently than typical ciphertexts (see Section 4.2.2 for details).

# 4 Security Analysis

## 4.1 Security Assumption

We begin by proving the IND-CPA security of the Zem.PKE scheme.

### 4.1.1 IND-CPA Security of Zem.PKE

**Theorem 1.** *Let $H$, $G$, and $\mathcal{X}$ be random oracles. For any IND-CPA adversary $\mathcal{A}$ against* Zem.PKE*, there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ with running time comparable to that of $\mathcal{A}$ such that:*

$$\mathrm{Adv}^{\text{IND-CPA}}_{\text{Zem.PKE}}(\mathcal{A}) \leq 2 \cdot \mathrm{Adv}^{\text{M-LWE}}_{n,q,k,B_{\eta_s},B_{\eta_e},k+1}(\mathcal{B}) + \mathrm{Adv}^{\text{PRF}}_{\mathcal{P}}(\mathcal{C}).$$

*Proof.* Let $\mathcal{A}$ be an adversary in the IND-CPA game, denoted by $G_0$. We define a sequence of games to progressively reduce the original security claim to the hardness of the M-LWE problem and the security of the PRF.

**Game $G_1$:** This game is identical to $G_0$ except that the secret and error vectors in Zem.PKE are sampled directly from $B_{\eta_s}$ and $B_{\eta_e}$. By the standard difference lemma, we have:

$$|\mathrm{Adv}_{G_0}(\mathcal{A}) - \mathrm{Adv}_{G_1}(\mathcal{C})| \leq \mathrm{Adv}^{\text{PRF}}_{\mathcal{P}}(\mathcal{C}),$$

where $\mathcal{C}$ runs in time comparable to $\mathcal{A}$.

**Game $G_2$:** This game is identical to $G_1$, except that the public key vector $\boldsymbol{b}$ is replaced with a uniformly random vector. By the difference lemma, it holds that:

$$|\mathrm{Adv}_{G_1}(\mathcal{C}) - \mathrm{Adv}_{G_2}(\mathcal{B})| \leq \mathrm{Adv}^{\text{M-LWE}}_{n,q,k,B_{\eta_s},B_{\eta_e},k}(\mathcal{B}) \leq \mathrm{Adv}^{\text{M-LWE}}_{n,q,k,B_{\eta_s},B_{\eta_e},k+1}(\mathcal{B}).$$

**Game $G_3$:** This game differs from $G_2$ in that the ciphertext is now sampled uniformly at random. Again, by the difference lemma, we have:

$$|\mathrm{Adv}_{G_2}(\mathcal{B}) - \mathrm{Adv}_{G_3}(\mathcal{B})| \leq \mathrm{Adv}^{\text{M-LWE}}_{n,q,k,B_{\eta_s},B_{\eta_e},k+1}(\mathcal{B}).$$

Since in $G_3$ there is no of the secret information, it follows that: $\mathrm{Adv}_{G_3}(\mathcal{B}) = 0$. Applying the triangle inequality across all games yields the claimed bound. □

### 4.1.2 IND-CCA Security of Zem.KEM in the ROM

Recall that Zem.KEM is derived from Zem.PKE by applying the $\mathrm{FO}^{\perp}_m$ transformation. The following analysis relies on a lemma established in [28].

**Lemma 3** (Proved in [28])**.** *Let* $\Pi$ *be a KEM obtained by applying the* $\mathrm{FO}^{\perp}_m$ *transform to a* $\gamma$*-spread encryption scheme* $\Sigma$*. Let* $q_D$ *denote the number of decapsulation oracle queries and* $q_{\mathrm{RO}}$ *the total number of queries to the random oracles* $G$*,* $H$*, and* $\mathcal{X}$*. Then, in the random oracle model, there exist adversaries* $\mathcal{A}$*,* $\mathcal{B}$*, and* $\mathcal{C}$ *with identical computational complexity such that:*

$$\mathrm{Adv}^{\text{IND-CCA}}_{\Pi}(\mathcal{A}) \leq 3 \cdot \mathrm{Adv}^{\text{IND-CPA}}_{\Sigma}(\mathcal{B}) + \mathrm{Adv}^{\text{FFP-CCA}}_{\Sigma^G}(\mathcal{C}) + \frac{2(q_{\mathrm{RO}} + q_D + 1)}{2^{256}} + q_D \cdot 2^{-\gamma}.$$

Theorem 2 bounds the advantage of an IND-CCA adversary against Zem.KEM in the random oracle model (ROM).

**Theorem 2.** *Let* Zem.PKE *be a* $\gamma$*-spread encryption scheme, and let* $G$*,* $H$*, and* $\mathcal{X}$ *be random oracles. Suppose that an IND-CCA adversary* $\mathcal{A}$ *makes at most* $q_{\mathrm{RO}}$ *queries to the random oracles and* $q_D$ *decapsulation oracle queries. Then, there exist adversaries* $\mathcal{B}$*,* $\mathcal{C}$*, and* $\mathcal{D}$ *with computational complexity comparable to that of* $\mathcal{A}$ *such that:*

$$\mathrm{Adv}^{\text{IND-CCA}}_{\text{Zem.KEM}}(\mathcal{A}) \leq 6 \cdot \mathrm{Adv}^{\text{M-LWE}}_{n,q,k,B_{\eta_s},B_{\eta_e},k+1}(\mathcal{B}) + 3 \cdot \mathrm{Adv}^{\text{PRF}}_{\mathcal{P}}(\mathcal{C}) + (q_D + 1) \cdot \mathrm{Adv}^{\text{FFP-CPA}}_{\text{Zem.PKE}^G}(\mathcal{D}) + \frac{2(q_{\mathrm{RO}} + q_D + 1)}{2^{256}} + 2q_D \cdot 2^{-\gamma}.$$

*Proof.* The bound follows by combining Lemma 3, Theorem 1, and inequality (1). □

To make full use of the result in Theorem 2, it is necessary to estimate the values of $\gamma$ and $\mathrm{Adv}^{\text{FFP-CPA}}_{\text{Zem.PKE}^G}(\mathcal{D})$. The parameter $\gamma$ is computed for all parameter sets in Section 4.1.4, while the estimation of the advantage $\mathrm{Adv}^{\text{FFP-CPA}}_{\text{Zem.PKE}^G}(\mathcal{D})$ is addressed in Section 4.1.5. Alternatively, one can adopt a conservative estimate of the advantage $\mathrm{Adv}^{\text{FFP-CCA}}_{\text{Zem.PKE}^G}(\mathcal{D})$ in the random oracle model. This leads to the following conservative bound.

**Theorem 3** (Conservative)**.** *Let* Zem.PKE *be a $\gamma$-spread and $\delta$-correct encryption scheme, and let $G$, $H$, and $\mathcal{X}$ be random oracles. Let $\mathcal{A}$ be an IND-CCA adversary (in the ROM) against* Zem.KEM*, making at most $q_{\mathrm{RO}}$ queries to the random oracles and $q_D$ decapsulation oracle queries. Then there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ with computational complexity comparable to that of $\mathcal{A}$ such that:*

$$\begin{aligned}\mathrm{Adv}^{\text{IND-CCA}}_{\text{Zem.KEM}}(\mathcal{A}) \leq\ & 6 \cdot \mathrm{Adv}^{\text{M-LWE}}_{n,q,k,B_{\eta_s},B_{\eta_e},k+1}(\mathcal{B}) + 3 \cdot \mathrm{Adv}^{\mathrm{PRF}}_{\mathcal{P}}(\mathcal{C}) \\ & + \frac{2(q_{\mathrm{RO}} + q_D + 1)}{2^{256}} + q_{\mathrm{RO}} \cdot \delta + q_D \cdot 2^{-\gamma}.\end{aligned}$$

*Proof.* The result follows by combining Lemma 3, Theorem 1, and inequality (3). □

### 4.1.3 IND-CCA security of Zem.KEM in the extended QROM

In this section, we present security bounds for an adversary attacking the proposed KEM in the IND-CCA game within the extended quantum random oracle model (QROM). Analogously to our ROM analysis, we provide two bounds — one of which is conservative. The following bounds are based on the lemma established in [28].

**Lemma 4** (Proved in [28])**.** *Let a KEM $\Pi$ be obtained by applying the $\mathrm{FO}^{\perp}_m$ transform to a $\gamma$-spread encryption scheme $\Sigma$. Let $q_D$ denote the number of decapsulation oracle queries, and let $q_{\mathrm{RO}}$ denote the number of quantum queries to the random oracles $G$, $H$, and $\mathcal{X}$. Then, in the* $\mathrm{eQROM}_{\mathrm{Enc}}$ *model, there exist adversaries $\mathcal{A}$, $\mathcal{B}$, and $\mathcal{C}$ such that the following inequality holds:*

$$\begin{aligned}\mathrm{Adv}^{\text{IND-CCA}}_{\Pi}(\mathcal{A}) \leq\ & 4\sqrt{(q_{\mathrm{RO}} + q_D) \cdot \mathrm{Adv}^{\text{IND-CPA}}_{\Sigma}(\mathcal{B})} + \mathrm{Adv}^{\text{FFP-CCA}}_{\Sigma^G}(\mathcal{C}) \\ & + \frac{8(q_{\mathrm{RO}} + q_D)}{2^{128}} + \theta_\gamma,\end{aligned}$$

*where*

$$\theta_\gamma \leq 12 q_D (q_{\mathrm{RO}} + 4 q_D) \cdot 2^{-\gamma/2}.$$

*Moreover, the computational complexity of $\mathcal{B}$ exceeds that of $\mathcal{A}$ by at most $O(q_{\mathrm{RO}} + q_D)$, while that of $\mathcal{C}$ exceeds it by at most $O(q_D)$.*

Theorem 4 provides a bound on the advantage of an IND-CCA adversary in the extended quantum random oracle model $\mathrm{eQROM}_{\mathrm{Enc}}$.

**Theorem 4.** *Let* Zem.PKE *be a $\gamma$-spread encryption scheme, and let $\mathcal{A}$ be an IND-CCA adversary in the* $\mathrm{eQROM}_{\mathrm{Enc}}$ *model against* Zem.KEM*, making at most $q_{\mathrm{RO}}$ queries to the random oracles $G$, $H$, and $\mathcal{X}$, and at most*

*$q_D$ decapsulation queries. Then there exist adversaries $\mathcal{B}$, $\mathcal{C}$, and $\mathcal{D}$ in the* $\mathrm{eQROM}_{\mathrm{Enc}}$ *model such that:*

$$\mathrm{Adv}^{\text{IND-CCA}}_{\text{Zem.KEM}}(\mathcal{A}) \leq 4 \cdot \sqrt{(q_{\mathrm{RO}} + q_D) \cdot \left(2 \cdot \mathrm{Adv}^{\text{M-LWE}}_{n,q,k,B_{\eta_s},B_{\eta_e},k+1}(\mathcal{B}) + \mathrm{Adv}^{\mathrm{PRF}}_{\mathcal{P}}(\mathcal{C})\right)} + \frac{8(q_{\mathrm{RO}} + q_D)}{2^{128}} + (q_D + 1) \cdot \mathrm{Adv}^{\text{FFP-CPA}}_{\text{Zem.PKE}^G}(\mathcal{D}) + \varepsilon_\gamma,$$

*where the term $\varepsilon_\gamma$ is bounded by*

$$\varepsilon_\gamma \leq 24 q_D (q + 4q_D) \cdot 2^{-\gamma/2}.$$

*Moreover, the running times of adversaries $\mathcal{B}$ and $\mathcal{C}$ satisfy*

$$\mathrm{Time}(\mathcal{B}) = \mathrm{Time}(\mathcal{C}) = \mathrm{Time}(\mathcal{A}) + O(q + q_D),$$

*and*

$$\mathrm{Time}(\mathcal{D}) = \mathrm{Time}(\mathcal{A}) + O(q_D).$$

*Proof.* The result follows directly from a combination of Lemma 4, Theorem 1, and inequality (2). □

Finally, we present a conservative bound on the advantage of an IND-CCA adversary in the extended quantum random oracle model.

**Theorem 5** (Conservative)**.** *Let* Zem.PKE *be a $\gamma$-spread and $\delta$-correct encryption scheme. Let $\mathcal{A}$ be an IND-CCA adversary in the* $\mathrm{eQROM}_{\mathrm{Enc}}$ *model against* Zem.KEM*, making at most $q_{\mathrm{RO}}$ queries to the random oracles $G$, $H$, and $\mathcal{X}$, and at most $q_D$ decapsulation queries. Then there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ in the* $\mathrm{eQROM}_{\mathrm{Enc}}$ *such that:*

$$\mathrm{Adv}^{\text{IND-CCA}}_{\text{Zem.KEM}}(\mathcal{A}) \leq 4 \cdot \sqrt{(q_{\mathrm{RO}} + q_D) \cdot \left(2 \cdot \mathrm{Adv}^{\text{M-LWE}}_{n,q,k,B_{\eta_s},B_{\eta_e},k+1}(\mathcal{B}) + \mathrm{Adv}^{\mathrm{PRF}}_{\mathcal{P}}(\mathcal{C})\right)} + \frac{8(q_{\mathrm{RO}} + q_D)}{2^{128}} + 10(q_{\mathrm{RO}} + q_D + 1)^2 \cdot \delta + \theta_\gamma,$$

*where the term $\theta_\gamma$ is bounded by*

$$\theta_\gamma \leq 12 q_D (q_{\mathrm{RO}} + 4q_D) \cdot 2^{-\gamma/2}.$$

*The running times of the adversaries satisfy:*

$$\mathrm{Time}(\mathcal{B}) = \mathrm{Time}(\mathcal{C}) = \mathrm{Time}(\mathcal{A}) + O(q + q_D).$$

*Proof.* The theorem follows from Lemma 4, Theorem 1, and inequality (4). □

### 4.1.4 The spreadness of Zem.PKE

In this section, we determine the concrete values of the spreadness parameter $\gamma$ for each proposed parameter set. We begin by presenting a general result for the LPR scheme [34] in the M-LWE setting. Recall that Zem.PKE can be viewed as the LPR scheme adapted to M-LWE and extended with compression and decompression operations.

**Theorem 6.** *Let the LPR scheme be instantiated in the M-LWE setting with ring $R = \mathbb{Z}[x]/(x^n + 1)$, where $n$ is a power-of-two, and let $k$, $q$, $\chi_s$, and $\chi_e$ denote the module dimension, modulus, and the distributions for secret and noise, respectively. Then the scheme is $\gamma$-spread with*

$$\gamma = \left( \max_{t \in \mathrm{supp}(\chi_e)} \Pr_{x \leftarrow \chi_e}[x = t] \right)^{n \cdot (k+1)}.$$

*Proof.* Fix a public key $pk = (\boldsymbol{A}, \boldsymbol{b})$ and message $m$. Let $c = (\boldsymbol{u}, v)$ be a fixed ciphertext corresponding to $m$. The probability $\Pr[\mathrm{Enc}(pk, m) = c]$ equals the probability of sampling vectors $\boldsymbol{r}$, $\boldsymbol{e}_1$, and $e_2$ such that:

$$\begin{cases} \boldsymbol{A}^T \cdot \boldsymbol{r} + \boldsymbol{e}_1 = \boldsymbol{u}, \\ \boldsymbol{b}^T \cdot \boldsymbol{r} + e_2 + \frac{q}{2} \cdot m = v. \end{cases}$$

Let $\overline{\boldsymbol{A}} = \begin{pmatrix} \boldsymbol{A}^T \\ \boldsymbol{b}^T \end{pmatrix}$ and $\overline{\boldsymbol{e}} = \begin{pmatrix} \boldsymbol{e}_1 \\ e_2 \end{pmatrix}$. Then the system above can be rewritten as a single matrix equation:

$$\overline{\boldsymbol{A}} \cdot \boldsymbol{r} + \overline{\boldsymbol{e}} = \begin{pmatrix} \boldsymbol{u} \\ v - \frac{q}{2} \cdot m \end{pmatrix}. \tag{6}$$

Hence, the desired probability is the probability of sampling $\boldsymbol{r}$ and $\overline{\boldsymbol{e}}$ such that equation (6) holds. Consider a collection of $M$ vectors $\boldsymbol{r}_1, \boldsymbol{r}_2, \ldots, \boldsymbol{r}_M$ and corresponding vectors $\overline{\boldsymbol{e}}_1, \overline{\boldsymbol{e}}_2, \ldots, \overline{\boldsymbol{e}}_M$ that satisfy (6). Then:

$$\max_{c \in \mathcal{C}} \Pr[\mathrm{Enc}(pk, m) = c] \leq \sum_{i=1}^{M} \Pr_{\boldsymbol{r} \leftarrow \chi_s^{n \times k}}[\boldsymbol{r} = \boldsymbol{r}_i] \cdot \Pr_{\overline{\boldsymbol{e}} \leftarrow \chi_e^{n \times (k+1)}}[\overline{\boldsymbol{e}} = \overline{\boldsymbol{e}}_i].$$

Each probability $\Pr[\overline{\boldsymbol{e}} = \overline{\boldsymbol{e}}_i]$ is upper-bounded by the probability of sampling a vector where all entries equal the most probable value of $\chi_e$. Thus, we obtain:

$$\begin{aligned} \max_{c \in \mathcal{C}} \Pr[\mathrm{Enc}(pk, m) = c] &\leq \left( \max_{x \in \mathrm{supp}(\chi_e)} \chi_e(x) \right)^{n(k+1)} \cdot \sum_{i=1}^{M} \Pr_{\boldsymbol{r} \leftarrow \chi_s^{n \times k}}[\boldsymbol{r} = \boldsymbol{r}_i] \\ &\leq \left( \max_{x \in \mathrm{supp}(\chi_e)} \chi_e(x) \right)^{n(k+1)}. \end{aligned}$$

Moreover, there exist specific choices of $pk$, $m$, and $c$ for which the bound above is tight. Let $\boldsymbol{A}$ be zero matrix, $\boldsymbol{b}$ be zero vector and let $m$ be any valid message. If the noise vectors $\boldsymbol{e}_1$ and $e_2$ consist entirely of the most probable value of $\chi_e$, then the ciphertext $c = (\boldsymbol{e}_1, e_2 + \frac{q}{2} \cdot m)$ is produced with probability:

$$\left( \max_{x \in \mathrm{supp}(\chi_e)} \chi_e(x) \right)^{n(k+1)}.$$

Therefore,

$$\max_{c \in \mathcal{C}} \Pr[\mathrm{Enc}(pk, m) = c] = \left( \max_{t \in \mathrm{supp}(\chi_e)} \Pr_{x \leftarrow \chi_e}[x = t] \right)^{n(k+1)}.$$

□

**The parameter $\gamma$ for Z512.** For the parameter set Z512, the value of $\gamma$ is 512.

*Proof.* In the Z512 parameter set, we have $d_u = 0$ and $d_v = 6$. Fix the first part of the ciphertext, i.e., the vector $\boldsymbol{u'}$. Since $d_u = 0$, the compression isn't applied to this part of ciphertext. Thus, for any given $\boldsymbol{r}$, there exists at most one vector $\boldsymbol{e}_1$ such that $\boldsymbol{u'} = \boldsymbol{A}^T \cdot \boldsymbol{r} + \boldsymbol{e}_1$.

Following the proof of Theorem 6, it implies that the probability of generating a ciphertext with a fixed $\boldsymbol{u'}$ is upper-bounded by $2^{-nk} = 2^{-512}$. This upper bound is tight, for instance, when the public matrix $\boldsymbol{A}$ is the zero matrix and $\boldsymbol{u'}$ is the zero vector.

Now consider the second part of the ciphertext — the compressed polynomial $v'$. Due to compression, a given $\boldsymbol{r}$ may correspond to multiple values of $e_2$ such that

$$\mathrm{Compress}_{d_v}(\boldsymbol{b}^T \cdot \boldsymbol{r} + e_2 + \frac{q}{2} \cdot m) = v'.$$

It can be shown that there exist choices of $pk$ and $m$ such that the value of $v'$ remains constant for all choices of $\boldsymbol{r}$ and $e_2$. This happens, for example, when $\boldsymbol{b}$ and $m$ are both zero. Therefore, the scheme is $\gamma$-spread with $\gamma = 512$. □

**The parameter $\gamma$ for Z768-R.** For the parameter set Z768-R, we have $\gamma = 1024$.

*Proof.* Since the Z768-R parameter set uses $d_u = d_v = 0$, Theorem 6 can be applied directly. □

**The parameter $\gamma$ for Z768-C.** For the parameter set Z768-C, we have $\gamma = 768 \cdot \log_2(4/3)$.

*Proof.* In the Z768-C parameter set, we have $d_u = 1$ and $d_v = 5$. Fix the first part of the ciphertext, i.e., the compressed vector $\boldsymbol{u'}$. We aim to compute the maximum probability of generating a noise vector $\boldsymbol{e}_1$ such that $\text{Compress}_{d_u}(\boldsymbol{A}^T \cdot \boldsymbol{r} + \boldsymbol{e}_1) = \boldsymbol{u'}$ for some fixed $\boldsymbol{r}$.

For any $1 \leq i \leq 768$, denote:

- $x_i$ as the $i$-th coordinate of $\boldsymbol{A}^T \cdot \boldsymbol{r}$,
- $e_i$ as the $i$-th coordinate of the noise vector $\boldsymbol{e}_1$,
- $u_i$ as the $i$-th coordinate of the compressed ciphertext $\boldsymbol{u'}$.

After adding noise and applying compression, $x_i$ is mapped to $u_i$ as follows:

1. If $e_i = -1$, then $u_i = \lfloor x_i/2 \rfloor$;
2. If $e_i = 0$, then $u_i = \lfloor (x_i + 1)/2 \rfloor$;
3. If $e_i = 1$, then $u_i = \lfloor (x_i + 2)/2 \rfloor$.

Observe that:

- For even $x_i$, both $e_i = -1$ and $e_i = 0$ result in $u_i = \lfloor (x_i + 1)/2 \rfloor$.
- For odd $x_i$, both $e_i = 0$ and $e_i = 1$ result in the same value.

Therefore, for each coordinate $i$, the probability that $u_i = \lfloor (x_i + 1)/2 \rfloor$ holds is $3/4$. Hence, for all 768 coordinates, the probability of obtaining a given $\boldsymbol{u'}$ (for fixed $\boldsymbol{r}$) is at most $(3/4)^{768} = 2^{-768 \cdot \log_2(4/3)}$. This upper bound is tight, for example, when $\boldsymbol{A}$ is the zero matrix and $\boldsymbol{u'}$ is the zero vector.

As in previous proofs, if the public vector $\boldsymbol{b}$ and the message $m$ are both zero, the second part of the ciphertext, $v'$, is constant (all zeros) regardless of $\boldsymbol{r}$ and $e_2$. We conclude that the Z768-C parameter set yields $\gamma = 768 \cdot \log_2(4/3)$. □

**The parameter $\gamma$ for Z1024.** For the parameter set Z1024, we have $\gamma = 1024$.

*Proof.* In the Z1024 parameter set, the values of $d_u$ and $d_v$ are the same as those used in the Z512 parameter set. Therefore, the value of $\gamma$ for Z1024 can be derived in exactly the same way as for Z512. □

### 4.1.5 The role of $\mathbf{Adv}^{\mathbf{FFP\text{-}CPA}}_{\mathbf{Zem.PKE}^G}(\mathcal{D})$

We intentionally leave the expression $\mathrm{Adv}^{\text{FFP-CPA}}_{\text{Zem.PKE}^G}(\mathcal{D})$ in explicit form and treat it as a separate assumption, as done in Theorems 2 and 4. The rationale behind this decision is that current estimates for this advantage are either conservative, as in [26, 30], or depend on additional, unquantified values, as in [28].

The advantage $\mathrm{Adv}^{\text{FFP-CPA}}_{\text{Zem.PKE}^G}(\mathcal{D})$ can be interpreted as the probability that an adversary $\mathcal{D}$—given only the public key—can generate a message that triggers a decryption failure. We examine this issue more thoroughly in Section 4.2.2.

In summary, overly conservative estimates are inconsistent with our current understanding of attacks that exploit decapsulation failures. Relying on such estimates leads to an inflated value of $\delta$, which in turn results in reduced efficiency. For this reason, we deviate from the standard treatment of $\delta$ when defining the parameter set Z768-R.

## 4.2 Cost of Known Attacks

Based on our security assumptions, there are two primary ways to attack Zemlyanika: (i) attacks on the underlying Module-LWE (M-LWE) problem, and (ii) attacks that exploit decapsulation failures.

### 4.2.1 Attacks Against the Underlying M-LWE Problem

At present, no known algorithm exploits the module structure of M-LWE to achieve a more efficient attack. Consequently, the best known attacks on M-LWE reduce to solving standard LWE instances using established techniques. There are two main strategies for attacking the LWE problem: the **primal** and the **dual** attacks.

**Primal Attack.** The goal of the primal attack is to recover the LWE secret $\boldsymbol{s}$. In its original form, the primal attack considers the lattice

$$\Lambda = \{\boldsymbol{y} \in \mathbb{Z}^m \mid \exists \boldsymbol{x} \in \mathbb{Z}^n : \boldsymbol{A}\boldsymbol{x} \equiv \boldsymbol{y} \mod q\},$$

where $\boldsymbol{A}$ is the public matrix and $n$, $q$, $m$ are the parameters of an LWE instance. The attack reduces to solving the Bounded Distance Decoding (BDD) problem in the lattice $\Lambda$, with the target point $\boldsymbol{b} = \boldsymbol{A} \cdot \boldsymbol{s} + \boldsymbol{e}$. That is, the goal is to find a lattice point $\boldsymbol{A} \cdot \boldsymbol{x}$ that is closest to $\boldsymbol{b}$. Modern variant of the primal attack considers the unique Shortest Vector Problem (uSVP), as introduced in [4]. This approach reduces BDD to a $\gamma$-uSVP instance by embedding $\Lambda$ into a higher-dimensional lattice $L$ that has a special structure

supporting the reduction. The basis matrix of the embedded lattice $L$ is constructed as:

$$B = \begin{pmatrix} \tilde{\boldsymbol{A}} & \boldsymbol{b} \\ \boldsymbol{0} & t \end{pmatrix},$$

where $t = d(\boldsymbol{b}, \Lambda)$ is the distance from $\boldsymbol{b}$ to the lattice $\Lambda$, and $\tilde{\boldsymbol{A}}$ is a basis of the $q$-ary lattice spanned by the columns of $\boldsymbol{A}$. It can be shown that if $t < \lambda_1(\Lambda)/(2\gamma)$, then $L$ contains a $\gamma$-unique shortest vector $\boldsymbol{v} = (\boldsymbol{e}, -t)$, whose first $m$ components correspond to the error vector $\boldsymbol{e}$.

**Dual Attack.** The dual attack targets the recovery of the LWE secret by searching for short vectors in the lattice

$$\Lambda = \{\boldsymbol{w} \in \mathbb{Z}^m \mid \boldsymbol{w} \cdot \boldsymbol{A} \equiv \boldsymbol{0} \mod q\},$$

which is the scaled dual (by $q$) of the lattice generated by the matrix $\boldsymbol{A}$. Let $\boldsymbol{v} \in \Lambda$ be a short vector. Consider the inner product $\langle \boldsymbol{v}, \boldsymbol{b} \rangle$, where $\boldsymbol{b}$ is either an LWE sample $\boldsymbol{b} = \boldsymbol{A} \cdot \boldsymbol{s} + \boldsymbol{e}$ or a uniformly random vector $\boldsymbol{b} \leftarrow \mathbb{Z}_q^m$. In the LWE case, we have:

$$\langle \boldsymbol{v}, \boldsymbol{b} \rangle \equiv \langle \boldsymbol{v}, \boldsymbol{e} \rangle \mod q,$$

since $\boldsymbol{v} \cdot \boldsymbol{A} \equiv \boldsymbol{0} \mod q$. It can be shown that $\langle \boldsymbol{v}, \boldsymbol{e} \rangle$ follows a Gaussian distribution over $\mathbb{Z}$, and thus $\langle \boldsymbol{v}, \boldsymbol{b} \rangle \mod q$ is likely to be small, given that both $\boldsymbol{v}$ and $\boldsymbol{e}$ have small norm. In contrast, when $\boldsymbol{b}$ is uniformly random in $\mathbb{Z}_q^m$, the inner product $\langle \boldsymbol{v}, \boldsymbol{b} \rangle$ is uniformly distributed over $\mathbb{Z}_q$, assuming $\boldsymbol{v} \neq \boldsymbol{0}$. Therefore, if we can find a sufficient number of short vectors $\boldsymbol{v} \in \Lambda$, we can use them to distinguish LWE samples from uniformly random ones.

**Other Attacks.** Algorithms for solving the LWE problem are not limited to the primal or dual attacks. However, many alternative approaches suffer from a critical limitation: they require an exponential number of LWE samples, which makes them impractical in our context. In particular, the underlying M-LWE problem in our construction provides only $n \cdot (k+1)$ samples, which significantly restricts the applicability of such algorithms. For example, the Auro-Ge algorithm [8] requires approximately $\exp\left(\frac{\pi}{4} \cdot nk\right)$ samples, and is therefore infeasible for our parameter sets. Similarly, algorithms based on the BKW paradigm [15, 3, 25, 31] also demand an exponential number of samples, making them unsuitable as well. Finally, exhaustive search over the secret space has time complexity $\omega(2^{kn})$, which is clearly prohibitive for practical parameters. Consequently, none of these alternative attacks pose a realistic threat to the security of Zemlyanika under the assumed hardness of the M-LWE problem.

**BKZ and the Core-SVP Model.** The core-SVP model, first introduced in [6], is a simplified yet conservative framework used to estimate the

security of lattice-based cryptosystems. In the context of LWE, the core-SVP model is employed to estimate the cost of both primal and dual attacks by analyzing the hardness of lattice reduction.

Both types of attacks rely on obtaining sufficiently short lattice vectors via basis reduction, typically performed using the BKZ algorithm [42]. The BKZ algorithm with block size $\beta$ calls an exact SVP oracle for sublattices with ranks up to $\beta$. Informally, increasing $\beta$ results in better reduction quality, but also higher computational cost.

To quantify the quality of lattice basis reduction, the *root-Hermite factor* $\delta_\beta$ is commonly used. The success of both primal and dual attacks depends on achieving a sufficiently small value of $\delta_\beta$. The required block size $\beta$ for reaching this target $\delta_\beta$ directly determines the cost of the attack. The core-SVP model simplifies the analysis by considering only the asymptotic complexity of a single SVP oracle call performed during the BKZ reduction process. These simplifications yield conservative yet meaningful estimates of cryptographic security.

Currently, the most efficient known algorithm for solving SVP in high-dimensional lattices is the sieving method. Its asymptotic complexity in a lattice rank $n$ is $2^{0.292n+o(n)}$, while the quantum version, leveraging Grover's search, reduces this to $2^{0.265n+o(n)}$. To estimate the required block size $\beta$ for successful attacks, we rely on the *Lattice Estimator* tool described in [5, 40] and available at `https://lattice-estimator.readthedocs.io/en/latest/readme_link.html`. Table 2 summarizes the estimated costs of known attacks against Zemlyanika KEM in the core-SVP model. Security levels are measured in bits, for both classical and quantum adversaries.

Table 2: Estimated security of Zemlyanika in the core-SVP model

| # | Primal (classical) | Dual (classical) | Primal (quantum) | Dual (quantum) |
|---|---|---|---|---|
| Z512 | 119 | 113 | 108 | 103 |
| Z768-R | 193 | 180 | 175 | 164 |
| Z768-C | 184 | 173 | 167 | 157 |
| Z1024 | 257 | 240 | 233 | 218 |

### 4.2.2 Attacks exploiting decapsulation failures

A decapsulation failure occurs when

$$\left\| \boldsymbol{e}^T \cdot \boldsymbol{r} - \boldsymbol{s}^T \cdot (\boldsymbol{e_1} + \boldsymbol{e_u}) + e_2 + e_v \right\|_\infty \geq \frac{q}{4},$$

where the vectors $\boldsymbol{s}$ and $\boldsymbol{e}$ are secret, while all other variables are known to the adversary. Since this failure condition depends on the secret key,

each failure potentially leaks partial information about it. Several works [23, 22, 20] have investigated attacks that exploit such failures. The original attack was introduced in [23], with subsequent improvements in [22, 20]. The general strategy proceeds in two main stages: first, obtaining ciphertexts that cause decapsulation failures; and second, using these ciphertexts to recover information about the secret key.

To reduce the number of decapsulation oracle queries needed to find ciphertexts that induce failures, the authors of [23] proposed a technique known as *failure boosting*. This technique relies on generating *weak* ciphertexts – those that cause a decapsulation failure with probability higher than average. By submitting only such weak ciphertexts to the oracle, the overall number of required queries can be significantly reduced. However, a successful attack on KEMs such as Saber or Kyber using the failure boosting technique still requires approximately $2^{131}$ decapsulation oracle queries.

Later, an improvement of the failure boosting technique, referred to as *directional failure boosting*, was introduced in [22]. This technique lowers the cost of identifying ciphertexts that trigger decapsulation failures by the use of information from previously identified failing ciphertexts. Based on their findings, the authors of [22] emphasize that, to mitigate attacks exploiting decryption failures, cryptographic scheme designers must ensure that an adversary cannot feasibly obtain even a single decapsulation failure.

Finding a decapsulation failure does not, by itself, compromise the security of the KEM. In [23], the authors proposed a method for estimating the secret key, assuming access to a sufficient number of ciphertexts that trigger decapsulation failure. The core idea is to estimate the secret and noise vectors in a way that reduces their variance, which makes the underlying LWE/LWR problem easier to solve. However, this approach relies on the adversary knowing which coefficients of the expression

$$\boldsymbol{e}^T \cdot \boldsymbol{r} - \boldsymbol{s}^T \cdot (\boldsymbol{e_1} + \boldsymbol{e_u}) + e_2 + e_v$$

exceed $q/4$ or fall below $-q/4$. However, work [23] does not provide a concrete method for reliably recovering this information.

To estimate the security against attacks that exploit decapsulation failures, we adopt a highly conservative assumption. Specifically, we assume that a *single* decapsulation failure is sufficient for a successful attack. This assumption aligns well with our security assumptions formalized in Theorems 2 and 4. Note that an adversary capable of successfully triggering a decapsulation failure effectively wins the FFP-CCA game, and vice versa. According to inequalities (1) and (2), the advantage in the FFP-CCA game can be estimated in terms of the advantage in the FFP-CPA game. Since

the parameter $\gamma$ is sufficiently small for all parameter sets, we assume in our analysis that

$$\mathrm{Adv}^{\text{FFP-CCA}}_{\text{PKE}^G} \approx (q_D + 1) \cdot \mathrm{Adv}^{\text{FFP-CPA}}_{\text{PKE}^G}.$$

The subsequent analysis can be viewed as an estimation of the term $\mathrm{Adv}^{\text{FFP-CPA}}_{\text{PKE}^G}(\mathcal{D})$.

If the M-LWE assumption holds, then the public key appears pseudorandom, and the adversary doesn't know even a bit of the secret key. Therefore, the adversary has two main strategies: generate a random message in the hope that it triggers a failure, or apply the failure boosting technique. In the first case, the probability of success is $\delta$. The second approach is more sophisticated and provides a higher success probability, which depends on the adversary's running time.

We present the failure boosting cost analysis only for the parameter set Z768-R. We demonstrate that failure boosting is impractical for Z768-R, which implies its infeasibility for the other parameter sets as well, since they exhibit significantly smaller values of $\delta$. We begin with the upper bound on the number of decapsulation queries $q_D$. According to the NIST guidelines, $q_D$ should not exceed $2^{64}$, as an honest user is unlikely to process a greater number of decapsulation requests. Consequently, a passive adversary must search for a ciphertext that triggers a decapsulation failure with probability at least $2^{-64}$.

For the parameter set Z768-R, we have $d_u = d_v = 0$, so the decapsulation failure condition becomes

$$||\boldsymbol{e}^T \cdot \boldsymbol{r} - \boldsymbol{s}^T \cdot \boldsymbol{e}_1 + e_2||_\infty \geq q/4,$$

where the vectors $\boldsymbol{r}$, $\boldsymbol{e}_1$, and the polynomial $e_2$ are known to the adversary. Recall that all entries of these vectors and polynomial are sampled from the distribution $B_1$. To simplify the analysis, we assume that the adversary can always choose $e_2$ to maximize the probability of failure. Under this assumption, the failure condition can be rewritten as:

$$||\boldsymbol{e}^T \cdot \boldsymbol{r} - \boldsymbol{s}^T \cdot \boldsymbol{e}_1||_\infty \geq q/4 - 1.$$

Then, for any message $m$, the probability that it causes a decryption failure is given by

$$p(m) = \Pr\left[||\boldsymbol{e}^T \cdot \boldsymbol{r} - \boldsymbol{s}^T \cdot \boldsymbol{e_1}||_\infty \geq q/4 - 1\right],$$

where $\boldsymbol{e} \leftarrow B^{n \cdot k}_{\eta_e}$ and $\boldsymbol{s} \leftarrow B^{n \cdot k}_{\eta_s}$, while all other vectors and polynomials are fixed and known to the adversary. Specifically, they are uniquely determined by the message $m$. The function $\alpha(f)$, defined as

$$\alpha(f) = \Pr_{m \leftarrow \mathcal{M}}\left[p(m) > f\right],$$

represents the probability that a randomly chosen plaintext encrypts to a ciphertext which causes decryption to fail with probability greater than $f \in (0, 1)$. The function $\beta(f)$,

$$\beta(f) = \frac{\sum\limits_{q>f} q \cdot \Pr\limits_{m \leftarrow \mathcal{M}}[p(m) = q]}{\alpha(f)},$$

denotes the expected decryption failure probability, conditioned on the event that this probability exceeds $f$.

The dependence $\beta(\alpha)$ for the parameter set Z768-R is presented in Figure 6. Currently, we estimate the advantage in the FFP-CPA game as $\beta$,

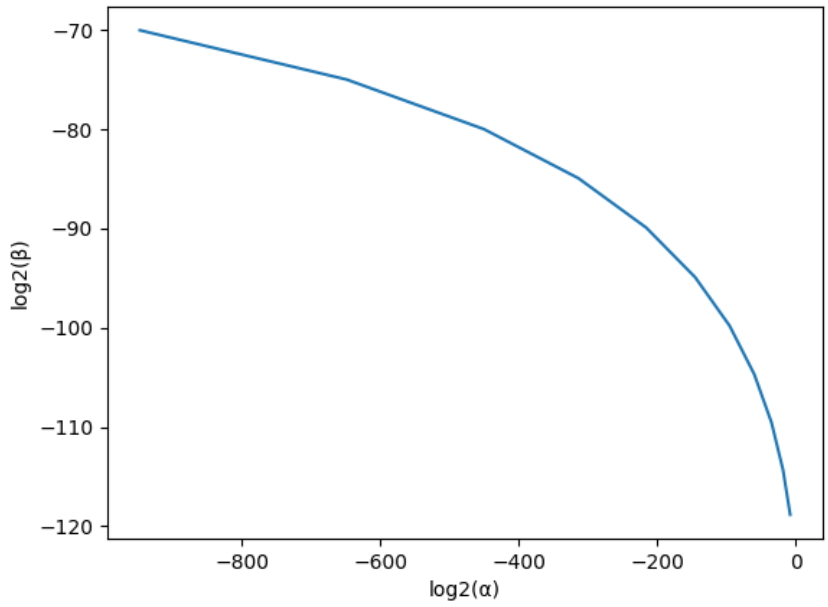


Figure 6: The dependence $\beta(\alpha)$ for the parameter set Z768-R

while the running time of the adversary $\mathcal{D}$ is given by $\text{Time}(\mathcal{D}) = \alpha^{-1}$ in the classical setting and $\text{Time}(\mathcal{D}) = \alpha^{-1/2}$ in the quantum setting due to the Grover's search. According to Figure 6, a successful attack using approximately $q_D = 2^{64}$ decapsulation queries would require more than $2^{800}$ operations on a classical computer and more than $2^{400}$ operations on a quantum computer. We therefore conclude that such an attack is infeasible in practice.

# 5 Implementation Details

We implement Zemlyanika in the `C` language, targeting general-purpose CPUs. In this section, we consider some implementation details.

**Symmetric primitives:** Symmetric primitives are used to implement the hash functions $H$ and $G$, the XOF $\mathcal{X}$, and the PRF $\mathcal{P}$. In our initial implementation, we use algorithms from the Keccak [13] family to allow for a fair comparison between Zemlyanika, Kyber, and Saber. The hash function $H$ is instantiated with SHA3-256, and $G$ is instantiated with SHA3-512. To implement the XOF and PRF, we use SHAKE with different security levels. Since $\mathcal{X}$ takes the public seed as input, we instantiate it with SHAKE128. On the other hand, $\mathcal{P}$ takes a secret 32-byte seed as a key and a nonce as input. We implement the function $\mathcal{P}$ acting on input $x$ with key $k_s$ as $\text{SHAKE256}(k_s \,||\, x)$.

**Toom-Cook multiplication:** It is well known that in structured LWE/LWR-based schemes, the performance of the polynomial multiplication algorithm significantly affects the overall performance of the scheme. Due to the use of a power-of-two modulus, we cannot employ NTT-based algorithms. Instead, we use a combination of Toom-Cook, Karatsuba, and schoolbook multiplication algorithms. Below, we briefly recall the details of the Toom-Cook algorithm.

Toom-Cook is an infinite family of algorithms, denoted as Toom-$n$ for $n \geq 2$, based on the divide-and-conquer paradigm. Suppose polynomials $a(x), b(x)$ have degree at most $m-1$, and that $n \mid m$. Letting $X = x^{m/n}$, we define the polynomials $A(X), B(X)$ as follows:

$$A(X) = a_0(x) + a_1(x)X + \cdots + a_{n-1}(x)X^{n-1},$$
$$B(X) = b_0(x) + b_1(x)X + \cdots + b_{n-1}(x)X^{n-1},$$

where each $a_i(x)$ and $b_j(x)$ has degree at most $m/n - 1$. The product $r(X) = A(X) \cdot B(X)$ then has degree at most $2n-2$, and its coefficients are polynomials in $x$ of degree at most $2m/n - 2$. From this perspective, the Karatsuba algorithm corresponds to the special case of Toom-2.

To compute $r(X)$, Toom-$n$ evaluates $A(X)$ and $B(X)$ at $2n-1$ distinct points, multiplies the resulting values pointwise, and then performs interpolation to recover $r(X)$. Therefore, Toom-$n$ consists of three main stages: evaluation, multiplication, and interpolation.

Let us assume that the evaluation points are fixed. The evaluation step is a linear transformation defined by the matrix $T_n$, and the interpolation step is determined by the inverse matrix $T_n^{-1}$. Thus, we have:

$$r(X) = T_n^{-1}\left(T_n(A(X)) \circ T_n(B(X))\right),$$

where $\circ$ denotes pointwise multiplication.

For some parameter sets, we use two layers of the Toom-Cook algorithm. We denote such an algorithm as Toom-$n$-$m$, which computes the product $r(X)$ as follows:

$$\begin{aligned} r(X) &= T_n^{-1}\left(T_m^{-1}\left(T_m(T_n(A(X))) \circ T_m(T_n(B(X)))\right)\right) \\ &= T_{n\text{-}m}^{-1}\left(T_{n\text{-}m}(A(X)) \circ T_{n\text{-}m}(B(X))\right). \end{aligned}$$

In Zemlyanika, polynomial multiplications only occur during the dot product computation. This property allows us to reduce the number of interpolation steps, exploiting the linearity of interpolation. Namely,

$$\begin{aligned} \sum_{i=1}^{k} A_i(X) \cdot B_i(X) &= \sum_{i=1}^{k} T^{-1}\left(T(A_i(X)) \circ T(B_i(X))\right) \\ &= T^{-1}\left(\sum_{i=1}^{k} T(A_i(X)) \circ T(B_i(X))\right), \end{aligned}$$

where $T$ is the linear transform associated with the Toom-Cook algorithm. This optimization was first proposed in [12].

Strictly speaking, the transformation $T_n$ is not a bijection, so its inverse $T_n^{-1}$ cannot be strictly defined. As a result, the output of the Toom-Cook algorithm is only accurate up to the $t$ most significant bits of each polynomial coefficient. The number of lost bits (i.e., the value of $t$) depends on the choice of evaluation points. Since polynomials in $R_q$ have degree at most 255, it is reasonable to use only Toom-$n$ algorithms with power-of-two $n$. We choose to use Toom-4 and Toom-2 (Karatsuba) algorithms. According to [16], the evaluation points used in Toom-4 are $\{0, \pm 1/2, \pm 1, 2, \infty\}$. For this choice of points, the precision loss corresponds to the 3 most significant bits.

For parameter sets Z512 and Z768-R, the polynomial multiplication algorithm is a combination of Toom-4-4 and School-book multiplication. Note that Toom-4-4 loses 6 bits of precision, allowing us to still use 16-bit integers for storing polynomial coefficients. The approximate cost of this multiplication method is equivalent to 49 school-book multiplications of polynomials of degree at most 15. Unfortunately, Toom-4-4 combined with 16-bit integers cannot be used for parameter sets Z768-C and Z1024 due to excessive precision loss. Instead, we use a combination of Toom-4-2-2 (i.e., Toom-4 followed by two layers of Karatsuba) and School-book multiplication. A similar approach is used in Saber [21]. This method loses only 3 bits of precision and incurs a computational cost roughly equivalent to 63 school-book multiplications of polynomials of degree at most 15.

**School-book multiplication:** For all parameter sets, we need to implement School-book multiplication for polynomials of degree at most 15. To reduce the number of cache misses, we use Algorithm 9. This optimization is achieved by ensuring sequential access to the elements of arrays storing polynomial coefficients, thereby improving cache performance.

**Algorithm 9** $c(x) :=$ Schb-mult$(a(x), b(x))$

```
1: for d := 0; d < 16; d := d + 1 do
2:     r := 0
3:     j := d
4:     for i := 0; j ≥ 0; i := i + 1 do
5:         r := r + a_i · b_j
6:         j := j − 1
7:     end for
8:     c_d := c_d + r
9: end for
10: for d := 16; d < 32; d := d + 1 do
11:     r := 0
12:     j := 15
13:     for i := d − 15; i ≤ 15; i := i + 1 do
14:         r := r + a_i · b_j
15:         j := j − 1
16:     end for
17:     c_d := c_d + r
18: end for
19: return c(x)
```

# 6 Performance Results and Comparison

This section presents the performance results of our implementation of Zemlyanika, along with a comparison of Zemlyanika's characteristics with those of Kyber [17] and Saber [21].

## 6.1 Performance Results

The performance results are shown in Table 3. All performance measurements were conducted on a single core of an Apple M1 processor, clocked at 3200 MHz. Execution times are given in microseconds. Each value in

the table represents the **median** of 1000 executions of the respective algorithm. The implementation was compiled using `clang` version 14.0.3 with the following compiler flags: `-O3 -fomit-frame-pointer -march=native`.

Table 3: Performance results for Zemlyanika

| # | KeyGen, $\mu$s | Encaps, $\mu$s | Decaps, $\mu$s |
|---|---|---|---|
| Z512 | 11.5 | 15.1 | 14.5 |
| Z768-R | 20.7 | 21.7 | 21.3 |
| Z768-C | 23.3 | 25.3 | 25.8 |
| Z1024 | 33.2 | 38.4 | 40.7 |

### 6.2 Comparison with Kyber and Saber

Here, we compare Zemlyanika with Kyber and Saber. The comparison is based on the following characteristics: key and ciphertext sizes, execution time, and security strength (in bits). The execution time was measured using the reference implementations of Kyber and Saber in the C language. We used the same benchmarking procedure, compiler, compilation flags, and CPU as for our own performance evaluation of Zemlyanika. According to the analysis from the Kyber specification [9], the number of security bits for Zemlyanika and Kyber is determined by the cost of the primal attack in the core-SVP model. In brief, the dual attack is not taken into account because the core-SVP model sufficiently underestimates its cost. The security strength against classical adversaries is indicated in the "cl." column, while the strength against quantum adversaries is shown in the "pq." column. The comparison results are summarized in Table 4.

## References


[1] Miklós Ajtai. The shortest vector problem in l2 is np-hard for randomized reductions (extended abstract). In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, STOC '98, page 10–19, New York, NY, USA, 1998. Association for Computing Machinery.

[2] Gorjan Alagic, David Cooper, Quynh Dang, Thinh Dang, John M. Kelsey, Jacob Lichtinger, and Yi-Kai Liu. Status report on the third round of the nist post-quantum cryptography standardization process, 2022.

Table 4: Comparison with Kyber and Saber

| # | $\|sk\|$, B | $\|pk\|$, B | $\|ct\|$, B | KeyGen, $\mu$s | Encaps, $\mu$s | Decaps, $\mu$s | cl. | pq. |
|---|---|---|---|---|---|---|---|---|
| Z512 | 832 | 672 | 768 | 11.5 | 15.1 | 14.5 | 119 | 108 |
| Kyber512 | 1632 | 800 | 768 | 21.2 | 24.5 | 28.0 | 118 | 107 |
| LightSaber | 1568 | 672 | 736 | 17.0 | 21.6 | 23.3 | 118 | 107 |
| Z768-R | 1216 | 992 | 1280 | 20.7 | 21.7 | 21.3 | 193 | 175 |
| Z768-C | 1408 | 1088 | 1152 | 23.3 | 25.3 | 25.8 | 184 | 167 |
| Kyber768 | 2400 | 1184 | 1088 | 33.1 | 37.8 | 42.9 | 182 | 165 |
| Saber | 2304 | 992 | 1088 | 31.4 | 37.9 | 32.3 | 189 | 172 |
| Z1024 | 1856 | 1440 | 1568 | 33.2 | 38.4 | 40.7 | 257 | 233 |
| Kyber1024 | 3168 | 1568 | 1568 | 50.8 | 54.4 | 61.0 | 256 | 232 |
| FireSaber | 3040 | 1312 | 1472 | 50.1 | 50.5 | 46.7 | 260 | 236 |


[3] Martin R. Albrecht, Carlos Cid, Jean-Charles Faugère, Robert Fitzpatrick, and Ludovic Perret. On the complexity of the bkw algorithm on lwe. *Designs, Codes and Cryptography*, 74(2):325–354, 2015.

[4] Martin R. Albrecht, Robert Fitzpatrick, and Florian Göpfert. On the efficacy of solving lwe by reduction to unique-svp. In Hyang-Sook Lee and Dong-Guk Han, editors, *Information Security and Cryptology – ICISC 2013*, pages 293–310, Cham, 2014. Springer International Publishing.

[5] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.

[6] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange: a new hope. In *Proceedings of the 25th USENIX Conference on Security Symposium*, page 327–343, Austin, USA, 2016. USENIX Association.

[7] Joël Alwen, Stephan Krenn, Krzysztof Pietrzak, and Daniel Wichs. Learning with rounding, revisited. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013*, pages 57–74, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[8] Sanjeev Arora and Rong Ge. New algorithms for learning in presence of errors. In Luca Aceto, Monika Henzinger, and Jiří Sgall, editors, *Automata, Languages and Programming*, pages 403–415, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[9] Roberto Avanzi, Joppe Bos, Leo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehle. Crystals-kyber: Nist post-quantum cryptography project submission package round 3, 2021.

[10] Paul Barrett. Implementing the rivest shamir and adleman public key encryption algorithm on a standard digital signal processor. In Andrew M. Odlyzko, editor, *Advances in Cryptology — CRYPTO' 86*, pages 311–323. Springer Berlin Heidelberg, 1987.

[11] Mihir Bellare and Phillip Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In Serge Vaudenay, editor, *Advances in Cryptology - EUROCRYPT 2006*, pages 409–426, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.

[12] Jose Maria Bermudo Mera, Angshuman Karmakar, and Ingrid Verbauwhede. Time-memory trade-off in toom-cook multiplication: an application to module-lattice based cryptography. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020(2):222–244, 2020.

[13] Guido Bertoni, Joan Daemen, Michael Peeters, and Gilles Van Assche. The keccak sha-3 submission, 2011.

[14] Nina Bindel, Mike Hamburg, Kathrin Hövelmanns, Andreas Hülsing, and Edoardo Persichetti. Tighter proofs of cca security in the quantum random oracle model. In Dennis Hofheinz and Alon Rosen, editors, *Theory of Cryptography*, pages 61–90, Cham, 2019. Springer International Publishing.

[15] Avrim Blum, Adam Kalai, and Hal Wasserman. Noise-tolerant learning, the parity problem, and the statistical query model. *J. ACM*, 50(4):506–519, 2003.

[16] Marco Bodrato and Alberto Zanoni. Integer and polynomial multiplication: towards optimal toom-cook matrices. page 17–24, New York, NY, USA, 2007. Association for Computing Machinery.

[17] Joppe Bos, Leo Ducas, Eike Kiltz, T Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehle. Crystals - kyber: A cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroSP)*, pages 353–367, 2018.

[18] Joppe W. Bos, Leo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehle. Crystals-dilithium: Nist post-quantum cryptography project submission package round 3, 2021.

[19] Jung Hee Cheon, Hyeongmin Choe, Dongyeon Hong, and MinJune Yi. Smaug: Pushing lattice-based key encapsulation mechanisms to the limits. In Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen, editors, *Selected Areas in Cryptography – SAC 2023*, pages 127–146, Cham, 2024. Springer Nature Switzerland.

[20] Jan-Pieter D'Anvers and Senne Batsleer. Multitarget decryption failure attacks and their application to saber and kyber. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *Public-Key Cryptography – PKC 2022*, pages 3–33, Cham, 2022. Springer International Publishing.

[21] Jan-Pieter D'Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In Antoine Joux, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *Progress in Cryptology – AFRICACRYPT 2018*, pages 282–305, Cham, 2018. Springer International Publishing.

[22] Jan-Pieter D'Anvers, Mélissa Rossi, and Fernando Virdia. (one) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020*, pages 3–33, Cham, 2020. Springer International Publishing.

[23] Jan-Pieter D'Anvers, Qian Guo, Thomas Johansson, Alexander Nilsson, Frederik Vercauteren, and Ingrid Verbauwhede. Decryption failure attacks on ind-cca secure lattice-based schemes. In *Public-Key Cryptography – PKC 2019*, volume 11443 of *Lecture Notes in Computer Science*, pages 565–598. Springer, 2019.

[24] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael Wiener, editor, *Advances in Cryptology — CRYPTO' 99*, pages 537–554, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.

[25] Qian Guo, Thomas Johansson, and Paul Stankovski. Coded-bkw: Solving lwe using lattice codes. In Rosario Gennaro and Matthew Robshaw,

editors, *Advances in Cryptology – CRYPTO 2015*, pages 23–42, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.

[26] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography*, pages 341–371, Cham, 2017. Springer International Publishing.

[27] Kathrin Hövelmanns. *Generic constructions of quantum-resistant cryptosystems*. Phd thesis, Ruhr-Universität Bochum, 2020.

[28] Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Failing gracefully: Decryption failures and the fujisaki-okamoto transform. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022*, pages 414–443, Cham, 2022. Springer Nature Switzerland.

[29] Kathrin Hövelmanns and Mikhail Kudinov. Treating dishonest ciphertexts in post-quantum kems – explicit vs. implicit rejection in the fo transform. In Ruben Niederhagen and Markku-Juhani O. Saarinen, editors, *Post-Quantum Cryptography*, pages 325–350, Cham, 2025. Springer Nature Switzerland.

[30] Kathrin Hövelmanns and Christian Majenz. A note on failing gracefully: Completing the picture for explicitly rejecting fujisaki-okamoto transforms using worst-case correctness. In Markku-Juhani Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography*, pages 245–265, Cham, 2024. Springer Nature Switzerland.

[31] Paul Kirchner and Pierre-Alain Fouque. An improved bkw algorithm for lwe with applications to cryptography and lattices. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology – CRYPTO 2015*, pages 43–62, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.

[32] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.

[33] Wenqi Liang, Zhaoman Liu, Xuyang Zhao, Yafang Yang, and Zhichuang Liang. Flexible and compact mlwe-based kem. *Mathematics*, 12(11), 2024.

[34] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *Advances in*

*Cryptology – EUROCRYPT 2010*, pages 1–23, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.

[35] Daniele Micciancio and Shafi Goldwasser. *Complexity of Lattice Problems: a cryptographic perspective*, volume 671 of *The Kluwer International Series in Engineering and Computer Science*. Kluwer Academic Publishers, Boston, Massachusetts, 2002.

[36] Arno Mittelbach and Marc Fischlin. *The Theory of Hash Functions and Random Oracles*. Springer, 2021.

[37] Peter L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44:519–521, 1985.

[38] National Institute of Standards and Technology (NIST). Module-lattice-based digital signature standard. Technical Report FIPS 204, U.S. Department of Commerce, 2023.

[39] National Institute of Standards and Technology (NIST). Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS 203, U.S. Department of Commerce, 2023.

[40] Rachel Player. *Parameter selection in lattice-based cryptography*. Phd thesis, Royal Holloway, University of London, 2018.

[41] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), 2009.

[42] C. P. Schnorr and M. Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66(1):181–199, 1994.

[43] Victor Shoup. Sequences of games: A tool for taming complexity in security proofs. *IACR Cryptology ePrint Archive*, 2004:332, 2004.

[44] Nigel P. Smart. *Cryptography Made Simple*. Springer, Cham, 1 edition, 2016.

[45] A. S. Zelenetsky and P. G. Klyucharev. Modular arithmetic optimization in kyber kem. *Journal of Computer Virology and Hacking Techniques*, 20(4):857–865, 2024.